MODERN APPLICATION DEVELOPMENT II

Previous Year Questions Concepts from Zero, then Every Answer

32 questions • JavaScript • Vue • Vuex • Vue Router • Flask • JWT • GraphQL • WebSockets • Git

How this paper is built – and how this book is built to match

PART A
JavaScript core

types and equality
references and copying
destructuring, spread
functions and "this"
classes, prototypes
reduce
promises, async/await
ES modules

PART B
Vue ecosystem

directives
lifecycle hooks
computed vs methods
Vue Router
Vuex store
SPA rendering
async data loading

PART C
Flask and backend

threading and timing
Flask-Caching
JWT structure
Flask-JWT-Extended
Flask-Security roles
401 vs 403
GraphQL

PART D
Platform, tooling

WebSocket handshake
cookies and
performance
Git workflow

PART E: answer key, traps, cheat sheets

EVERY concept is taught from zero BEFORE its question appears.

So read straight through. By the time you reach a question, you will already have the tool it tests.
Every JavaScript answer in this book was verified by actually running the code.

A previous-year paper is the single most valuable revision tool you have – but only if you understand *why* each answer is right. This book teaches the concept first, then solves the question, then shows you the trap that was set for you.

READ THIS FIRST

How to use this book

There are **32 questions** in this paper. None of them had an answer marked, so every single answer here was derived from first principles and then **checked**.

HOW THE ANSWERS WERE VERIFIED

- **All 8 pure-JavaScript output questions were actually executed** in Node.js, and the real console output is reproduced in this book. Those answers are not opinions.
- Framework behaviour (Vue lifecycle order, Vuex, Vue Router, Flask-Caching key rules, JWT structure, HTTP status codes, Git command semantics) was reasoned from documented behaviour and cross-checked against the option lists.
- Where a question is **ambiguously worded**, this is said openly and both readings are explained – rather than pretending the paper is perfect.

How to work through it

If you have time (recommended): read straight through from Part A. Each part teaches the concept from zero, with a story and a picture, and only then shows the question. You will find the question easy by the time you reach it – which is the whole point.

If you are revising the night before: read the answer key on the next page, then jump to the **Trap Museum** in Part E. It lists every trick this paper played, in one place. Then come back and read the sections for the traps you fell into.

Map of the book

Part	Topic	Questions covered
A1	Types, `typeof`, NaN, `==` vs `===`	Q10
A2	References, copying, shallow vs deep	Q20
A3	Destructuring, rest and spread	Q12
A4	Functions and the `this` keyword	Q1, Q21
A5	Classes, `extends`, prototype chain	Q11
A6	Array methods and `reduce`	Q2
A7	Promises, async/await, error handling	Q7, Q14
A8	ES modules: import and export	Q5

B1	Vue directives	Q15
B2	Vue lifecycle hooks	Q18
B3	Computed properties vs methods	Q28, Q29, Q30
B4	Vue Router: matching, wildcards, component reuse	Q9, Q31, Q32
B5	Vuex: state, mutations, actions	Q8, Q23
B6	SPA routing and async data loading	Q16, Q27
C1	Flask request threading and timing	Q22
C2	Flask-Caching `memoize`	Q13
C3	JWT and Flask-JWT-Extended	Q3, Q6, Q25
C4	Flask-Security, roles, 401 vs 403	Q19
C5	GraphQL	Q24
D1	WebSocket handshake	Q4
D2	Cookies and performance	Q17
D3	Git workflow commands	Q26
E	Answer key, Trap Museum, cheat sheets, practice	revision

The boxes you will see

STORY BOX

A real-life picture of the idea. Read this one first, always.

DEFINITION BOX

The proper technical wording, the one to write in an exam.

TRAP BOX

The exact mistake the examiner wanted you to make.

ANSWER BOX

The final answer, with verified output where applicable.

QUICK REFERENCE

Answer key at a glance

Questions are numbered in the order they appear in the paper. Verified answers are marked **[run]** where the code was actually executed.

Q	Topic	Answer
1	JS `this` keyword	Arrow functions do not have their own `this` and inherit it from the surrounding scope
2	Array `reduce`	**6** [run]
3	JWT vs sessions	JWTs are stateless and eliminate the need for server-side session storage
4	WebSocket handshake	**GET**
5	ES module import	`import { strikeRate, playerName } from "./battingStats.js";`
6	JWT payload	User-related data (claims) such as identity and expiration time
7	Promise settle once	**Success!** [run]
8	Vuex async action	`['Angular', 'Vue', 'React']`
9	Vue Router wildcard	Browser redirects to /about and displays "About"
10	typeof / NaN / equality	**number, object, false, false** [run]
11	Classes and prototypes	Dog Labrador barks / This is a Dog / true / true / true [run]
12	Destructuring and rest	John / `{ lastName:'Doe', age:30 }` / `{ firstName, location, lastName, age }` [run]
13	Flask-Caching memoize	**10 seconds**
14	Promise reject + await	**Failed [75,75,63]** [run]
15	Vue directives match	**1-B, 2-D, 3-C, 4-A**
16	SPA client-side routing	Reduced load on the backend server **+** Seamless page transitions without full reloads
17	Cookies and performance	Cookie size can impact performance **+** request to static.example.com will include the cookies
18	Vue lifecycle hooks	Status: Initial › Mounted › Ready › Done, **Value: 9**
19	Flask-Security roles	**200, 401, 403**
20	Shallow vs deep copy	**Ambrane 50 50 30** [run]
21	Arrow `this` capture	**100 undefined undefined** [run]
22	Flask threaded timing	**20, 40**
23	Vuex in a component	`this.$store.state.packed` and `this.$store.commit("ADD_ITEM", item)`
24	GraphQL	Mutations can change the data store **+** services are defined by types, fields and per-field functions
25	Flask-JWT-Extended	Returns "Welcome Jassi" only with a valid token in the Authorization header **+** no token gives an unauthorized error

26	Git commands	`git checkout -b` • `git add .` • `git commit -m` • `git push origin feature-branch`
27	Vue async render delay	Sidebar in an already-rendered parent **+** component waits for the API response **+** data fetched asynchronously after created
28	Computed run count	**1**
29	Computed re-evaluation	`bigNumber()` will run again because a dependency changed
30	Why computed, not method	Computed properties are cached based on their dependencies
31	Router component reuse	The route changes, the component is reused, and `updated()` is called
32	Why `updated()` fires	Because a watched route parameter updates parent reactive state, triggering a component update

PART A

JavaScript Core

Ten of the thirty-two questions are pure JavaScript. Nothing in this part assumes any prior knowledge – we start from what a value is.

Almost every JavaScript question in this paper is testing one of **four** underlying ideas. Once you see them as four ideas rather than ten unrelated puzzles, the whole part becomes predictable:

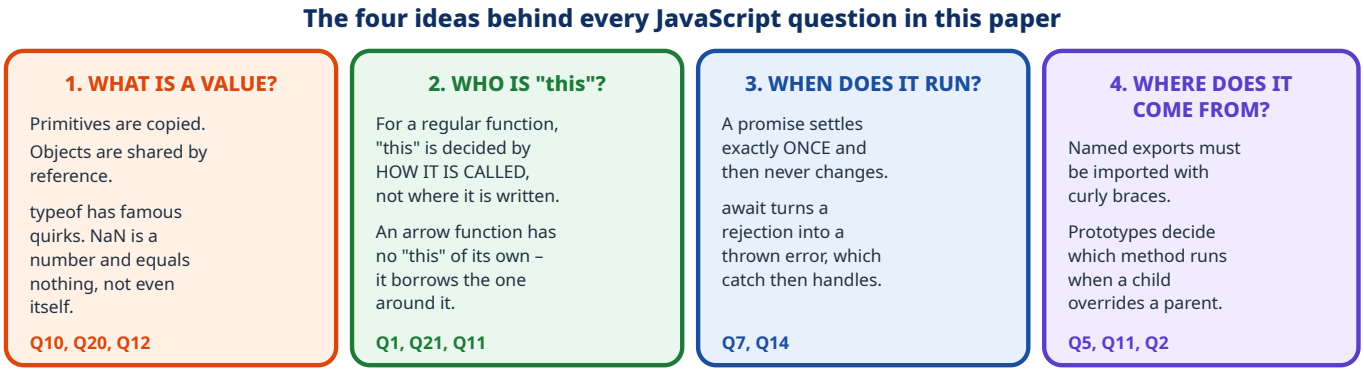

Figure A.0 – *Four questions to ask yourself. Almost every JS trap in this paper is one of them.*

PART A1 • QUESTION 10

Types, `typeof`, NaN and equality

A1.1 What is a "value" in JavaScript?

STORY BOX • THINGS AND ADDRESSES

Imagine a school. Some things you can write directly on a slip of paper: a number (**42**), a word (**"Ravi"**), a yes/no (**true**). If your friend wants it, you copy it onto their slip. Now there are **two independent copies**, and scribbling on yours does not change theirs.

Other things are too big for a slip – a whole classroom of furniture, say. So instead you write down the **room number**. If your friend copies that room number, you both hold a pointer to the **same room**. If you move a chair, your friend sees the chair move.

JavaScript works exactly like this. Small simple values are **primitives** (copied). Everything else is an **object** (shared by reference). This single distinction explains Q20 completely, and it is the most important idea in the language.

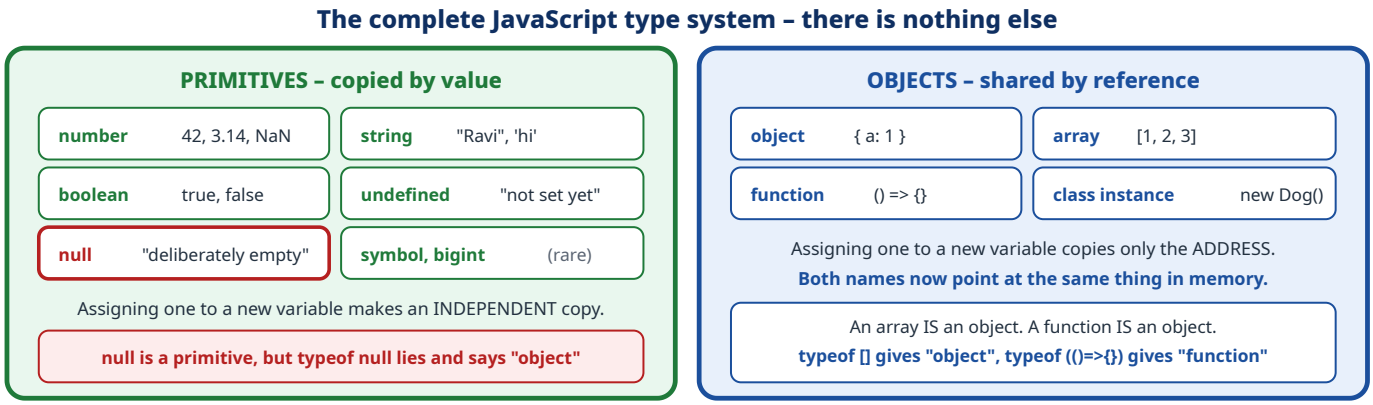

Figure A1.1 – *Two columns. Which column a value is in decides how it behaves when copied.*

A1.2 `typeof`, and its two famous lies

`typeof x` returns a **string** naming the type of `x`. It is mostly sensible, and has two well-known oddities that examiners love.

Expression	Result	Comment
typeof 42	"number"	as expected
typeof "hi"	"string"	as expected
typeof true	"boolean"	as expected
typeof undefined	"undefined"	as expected
typeof {}	"object"	as expected
typeof []	"object"	**Lie 1.** An array is an object. Use `Array.isArray([])` to test for arrays.
typeof null	"object"	**Lie 2.** `null` is a primitive, but this bug from 1995 can never be fixed without breaking the web. **Memorise it.**
typeof NaN	"number"	**Surprising but logical.** NaN means "Not a Number", yet it lives in the number type – it is the number type's way of saying "this calculation failed".
typeof function(){}	"function"	the one object that gets its own name

STORY BOX • WHY NAN IS A "NUMBER"

Think of a calculator. You type `0 ÷ 0` and the screen shows **Error**. That **Error** appears *on the number display* – it is the number display's way of reporting a failed sum. It has not turned your calculator into a word processor.

`NaN` is that **Error** on the number display. It is produced by arithmetic, it lives in arithmetic, so its *type* is `number`. The name "Not a Number" describes its **meaning**, not its type.

A1.3 `==` versus `===`, and the strangest value in the language

STORY BOX • THE STRICT EXAMINER AND THE LENIENT ONE

Two examiners check whether two answers "match".

The strict one (`===`) says: the answers must be the **same type** and the **same value**. "5" written as a word is not the same as the digit 5. No conversions allowed.

The lenient one (`==`) says: let me first **convert** them to a common form, then compare. So `"5" == 5` is true, because the string is converted to a number first.

Because the lenient examiner's conversion rules are long and surprising, professional advice is simple: **always use `===`**. Use `==` only if you deliberately want the one useful case, `null == undefined`.

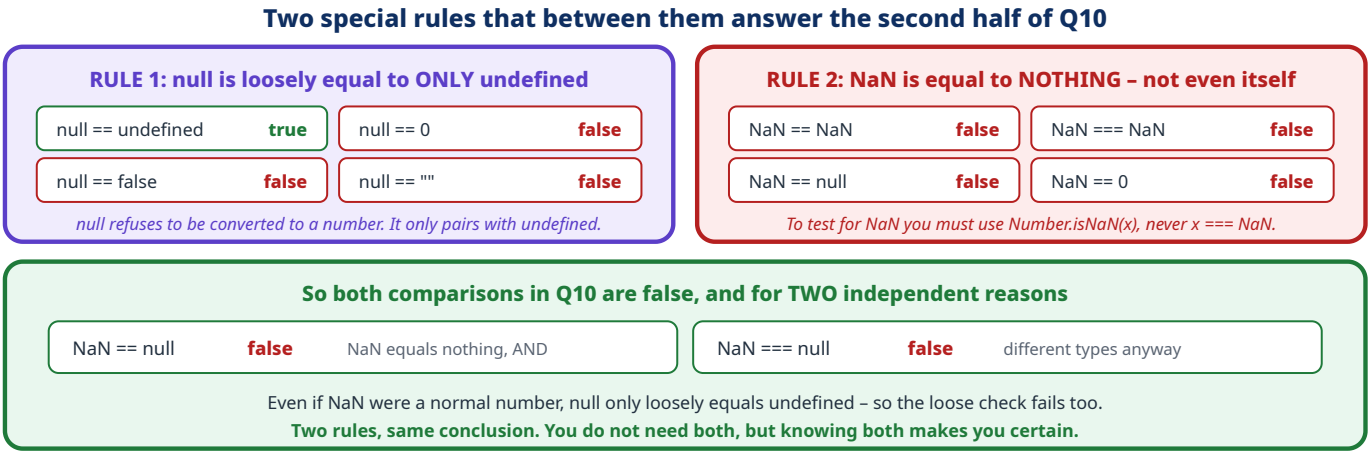

Figure A1.2 – *The two equality rules that decide Q10.*

Question 10

JavaScript output

What will be the output of the following JavaScript code?

```
console.log(typeof NaN);
console.log(typeof null);
console.log(NaN == null);
console.log(NaN === null);
```

A. number, object, false, false

B. NaN, null, true, true

C. number, object, true, false

D. NaN, null, false, false

Line-by-line solution

Line	Code	Output	Reason
1	typeof NaN	**number**	NaN is the number type's error value, so its type is `number`.
2	typeof null	**object**	The historical `typeof` bug. `null` is really a primitive, but `typeof` reports `"object"`.
3	NaN == null	**false**	NaN is not equal to anything, and `null` is loosely equal only to `undefined`.
4	NaN === null	**false**	Strict equality also requires the same type, and `number` is not `object`.

Verified output

```
number | object | false | false
```

Produced by actually running the four lines in Node.js.

ANSWER TO QUESTION 10

number, object, false, false (option A)

Why each wrong option is tempting

Option	The misunderstanding it is fishing for
B	Thinking `typeof` returns the **value** rather than the type name, and that two "empty-ish" values must be equal. Both halves are wrong.
C	The most dangerous distractor. It gets both `typeof` answers right, then assumes `NaN == null` is true because `==` "is loose". This is the option you pick if you know `null == undefined` is true and over-generalise it.
D	Gets the equality half right but still thinks `typeof` echoes the value.

TRAP BOX • THE THREE FACTS TO MEMORISE COLD

1. `typeof null` is **"object"** – a known language bug.
2. `typeof NaN` is **"number"**.
3. `null` is loosely equal to **undefined and nothing else**; `NaN` is equal to **nothing at all**, including itself.

EXTRA MARKS: RELATED FACTS EXAMINERS PAIR WITH THIS

null == undefined	true	the one useful loose comparison
null === undefined	false	different types
Number.isNaN(NaN)	true	the correct way to test for NaN
[] == false	true	loose equality converts both to 0
0.1 + 0.2 === 0.3	false	floating point, a classic follow-up

PART A2 • QUESTION 20

References, copying, shallow vs deep

This is the single most examined idea in JavaScript, and Q20 is a beautifully designed test of it. Take it slowly.

A2.1 Assignment does not copy an object

STORY BOX • TWO NAMES FOR ONE HOUSE

A house has the address **12 Gandhi Road**. Ravi writes that address in his diary, and Sita copies the same address into hers.

Now Ravi paints the front door red. When Sita visits her address, she sees a **red door** – because there was only ever **one house**. Two diaries, two pieces of paper, one house.

`const ref1 = item` copies the **address**, not the house. So `ref1.name = "Ambrane"` repaints the one and only object, and `item.name` changes too.

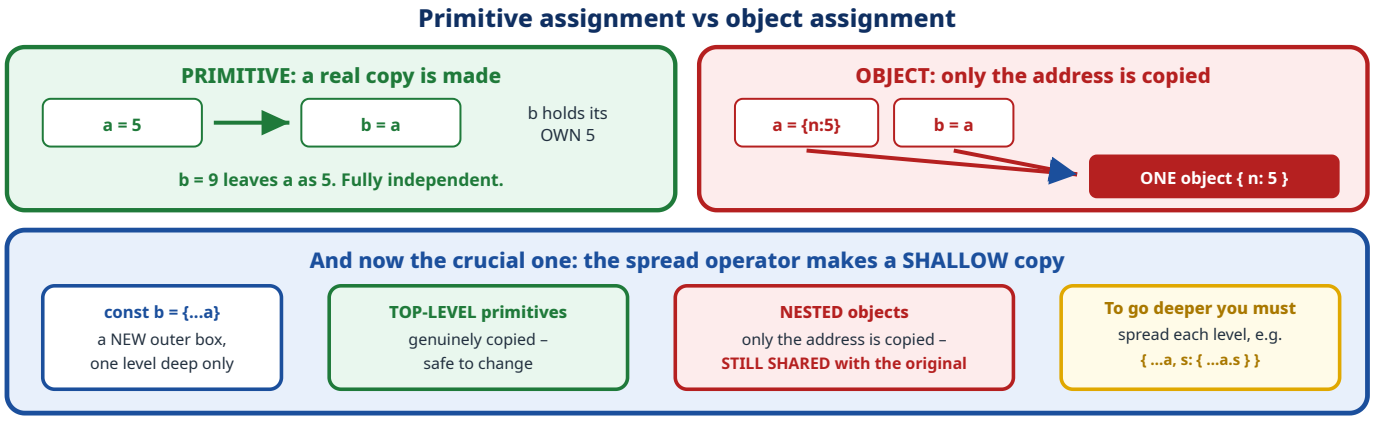

Figure A2.1 – *"Shallow" means **one level**. Everything nested inside remains shared.*

A2.2 The three kinds of copy, drawn on one memory map

Q20 creates three references, each a different kind. Here is what memory actually looks like **before** any modification:

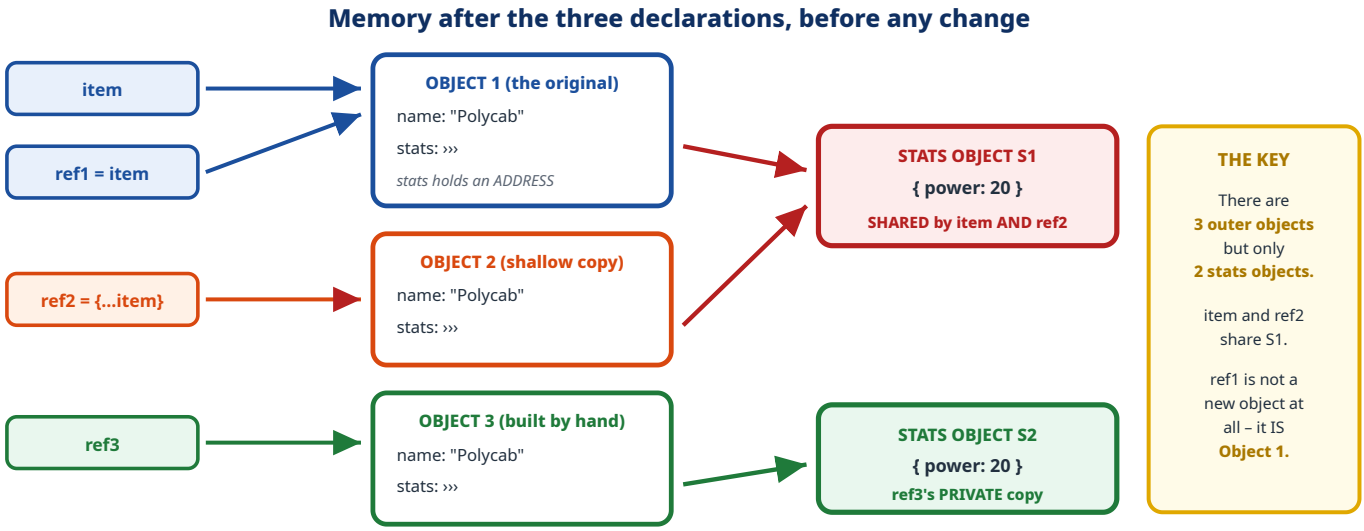

Figure A2.2 – *Count the boxes: **3 outer objects, 2 inner objects**. That count is the whole answer.*

Question 20

JavaScript output

Consider the following code snippet running in a browser. What will be the output on the console?

```
function inventory() {
  const item = { name: "Polycab", stats: { power: 20 } };

  const ref1 = item;                          // same object
  const ref2 = { ...item };                   // shallow copy
  const ref3 = { name: item.name, stats: { ...item.stats } }; // deep-ish copy

  ref1.name = "Ambrane";
  ref2.stats.power = 50;
  ref3.stats.power = 30;

  console.log(item.name);
  console.log(item.stats.power);
  console.log(ref2.stats.power);
  console.log(ref3.stats.power);
}
inventory();
```

A. Ambrane 20 50 30

B. Polycab 50 50 30

C. Ambrane 50 50 30

D. Ambrane 50 30 30

Step-by-step solution

1 **`ref1.name = "Ambrane"`**
`ref1` is not a copy – it **is** Object 1. Setting its `name` sets Object 1's name.
Therefore `item.name` becomes "Ambrane".

2 **`ref2.stats.power = 50`**
Read it in two hops. First `ref2.stats` – that follows the address to **S1**, the *shared* stats object, because a spread only copied the address. Then `.power = 50` writes into S1.
So `item.stats.power` also becomes 50, even though we never mentioned `item`. This is the trap.

3 **`ref3.stats.power = 30`**
`ref3.stats` points to **S2**, a brand-new object created by `{ ...item.stats }`. Writing to it affects **nobody else**.
Only `ref3.stats.power` becomes 30.

4 **Now read the four logs**

Expression	Value	Why
item.name	**Ambrane**	changed via ref1, the same object
item.stats.power	**50**	changed via ref2, because S1 is shared
ref2.stats.power	**50**	the same S1, so the same value
ref3.stats.power	**30**	its own private S2

Verified output

```
Ambrane
50
50
30
```

ANSWER TO QUESTION 20

Ambrane 50 50 30 (option C)

Why each wrong option is tempting

Option	The misunderstanding it is fishing for
A	**Ambrane 20** 50 30 – you understood that `ref1` shares the object, but believed `{...item}` makes a *full* copy, so you thought `item.stats` was protected. This is the classic "I forgot spread is shallow" answer.
B	**Polycab** 50 50 30 – the reverse mistake: you knew nested objects are shared, but thought `ref1 = item` creates a copy. It does not.
D	Ambrane 50 **30** 30 – you thought `ref3`'s change leaked back into `ref2`. It cannot: `ref3.stats` was explicitly built as a new object.

THE TWO-SECOND METHOD FOR ANY QUESTION OF THIS SHAPE

Draw the boxes and count them. Before tracing any assignment, sketch how many *distinct objects* exist and which variables point to each. Once the picture is right the answer falls out with no thinking. In this question: **3 outer, 2 inner**.

Then apply one rule: **a dot on the left of `=` writes into whatever object that path lands on.**

EXTRA MARKS: HOW TO MAKE A GENUINELY DEEP COPY

```
// shallow – nested objects still shared
const a = { ...original };

// deep – modern, built in, handles nesting and cycles
const b = structuredClone(original);

// deep – old trick, but loses functions, undefined and Date types
const c = JSON.parse(JSON.stringify(original));
```

PART A3 • QUESTION 12

Destructuring, rest and spread

A3.1 Destructuring is just unpacking

STORY BOX • UNPACKING THE GROCERY BAG

A bag arrives with **rice, dal, oil and salt**. You say: "take out the rice and the oil, and leave **everything else** in a smaller bag."

You now hold two named items and one bag containing the leftovers. Nothing was destroyed – the original bag is untouched. That is **destructuring with rest**:

```
const { rice, oil, ...leftovers } = bag;
```

DEFINITION BOX • THE THREE OPERATORS

- **Destructuring** – `const { a, b } = obj` pulls named properties out into variables. The variable takes the property's name unless you rename it with `{ a: newName }`.
- **Rest** (`...x` on the **left** of `=`) – *collects* all remaining properties into a new object. It only ever appears last.
- **Spread** (`...x` on the **right**, inside a new literal) – *pours* an existing object's properties into the one being built.

Same three dots, opposite jobs. Left of `=` gathers; right of `=` scatters.

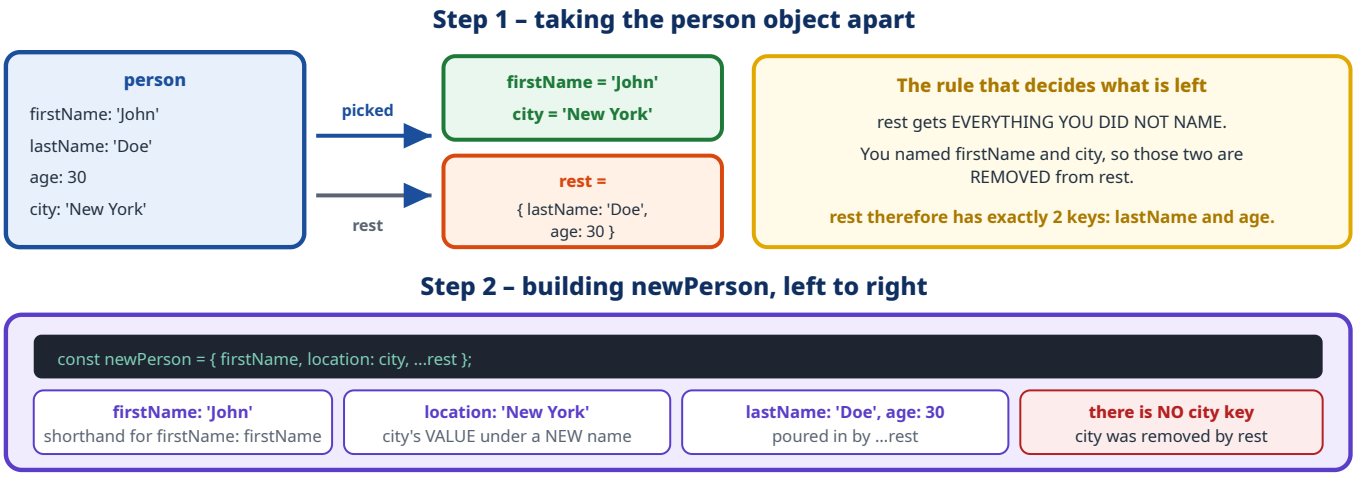

Figure A3.1 – *The critical consequence: because `city` was destructured out, `rest` does not contain it, so `newPerson` has `location` but **no** `city`.*

Question 12

JavaScript output

What will be the output of the above program?

```
const person = { firstName: 'John', lastName: 'Doe', age: 30, city: 'New York' };

const { firstName, city, ...rest } = person;
const newPerson = { firstName, location: city, ...rest };

console.log(firstName);
console.log(rest);
console.log(newPerson);
```

A. John / `{ city, lastName, age }` / `{ firstName, location, city, lastName, age }`

B. John / `{ lastName: 'Doe', age: 30 }` / `{ firstName: 'John', location: 'New York', lastName: 'Doe', age: 30 }`

C. John / `{ lastName, age }` / `{ firstName, location, city, lastName, age }`

D. undefined / ... / ...

Step-by-step solution

1 The destructuring line

`firstName` takes `'John'`. `city` takes `'New York'`. `rest` collects **everything not named**, which is `lastName` and `age` only.

`rest = { lastName: 'Doe', age: 30 }` – two keys, not three.

2 Building `newPerson`

Three pieces, in written order: `firstName: 'John'`, then `location: 'New York'` (the value of `city` stored under a new key), then the two keys spread from `rest`.

`{ firstName: 'John', location: 'New York', lastName: 'Doe', age: 30 }`

3 Why `city` does not appear in `newPerson`

This is the whole question. `city` was **pulled out** of the object during destructuring, so it is **not inside `rest`**. Spreading `rest` therefore cannot reintroduce it. The only trace of it is the `location` key, which carries its *value* under a different *name*.

Verified output

```
John
{ lastName: 'Doe', age: 30 }
{ firstName: 'John', location: 'New York', lastName: 'Doe', age: 30 }
```

ANSWER TO QUESTION 12

Option B

John • `{ lastName: 'Doe', age: 30 }` • `{ firstName: 'John', location: 'New York', lastName: 'Doe', age: 30 }`

Why each wrong option is tempting

Option	The misunderstanding it is fishing for
A	Puts `city` inside `rest`. That would be true only if you had **not** destructured `city` by name. Naming it removes it.
C	The most seductive option: it gets `rest` exactly right, then adds a `city` key to `newPerson` anyway. Ask yourself *where would that key come from?* Nothing in the code writes a `city` property.
D	Claims `firstName` is `undefined`. It is not – the property exists on `person`, so destructuring finds it.

TRAP BOX • RENAMING VS ADDING

`location: city` is **not** "add a location and keep the city". It reads `city`'s *value* and stores it under the key `location`. The old key name is gone. If the exam wanted both, the code would have had to say `{ ...rest, city, location: city }`.

EXTRA MARKS: TWO MORE DESTRUCTURING FACTS THAT GET ASKED

`const { a = 5 } = {}`	`a` is 5 – a **default value** is used when the key is missing
`const { a: { b } } = { a: { b: 1 } }`	**nested** destructuring; `b` is 1 and `a` is never created
`const [x, , z] = [1, 2, 3]`	**array** destructuring by position; the gap skips an element, so `z` is 3
`{ ...a, ...b }`	if both have the same key, **the later one wins**

PART A4 • QUESTIONS 1 AND 21

Functions and the `this` keyword

`this` confuses more learners than anything else in JavaScript, because people try to work out what it *should* mean instead of applying one mechanical rule. Learn the rule and it becomes easy.

A4.1 The one rule for a regular function

STORY BOX • "WHO IS SPEAKING?"

Imagine a written speech: **"I hereby declare this shop open."** The word **"I"** has no fixed meaning – it depends entirely on **who reads the speech out loud**. If the mayor reads it, "I" is the mayor. If a student reads it, "I" is the student. The paper does not decide; the **speaker at that moment** decides.

`this` inside a regular function is that word **"I"**. It is not fixed when the function is *written*. It is decided each time the function is **called**, by looking at **what is immediately to the left of the dot**.

THE RULE, IN FIVE WORDS

Look left of the dot.

`obj.fn()` › `this` is `obj`. `fn()` with nothing to the left › there is no owner, so `this` falls back to the global object (`window` in a browser), or `undefined` in strict mode and inside modules.

The four ways a regular function can be called, in priority order

1. NEW binding — `new Dog('Dog')` — "this" is the brand-new object being constructed

Highest priority. This is what happens in every class constructor.

2. EXPLICIT binding — `fn.call(obj)` — "this" is whatever you passed in

call, apply and bind let you force it by hand.

3. METHOD binding — `obj.fn()` — **"this" is obj – the thing LEFT OF THE DOT**

The common case, and the one Q21 tests. The dot is doing all the work.

4. DEFAULT binding — `fn()` — **nothing left of a dot, so there is NO owner**

Non-strict browser code: "this" becomes **window**. Strict mode or a module: **undefined**.

Either way, window.value is undefined – which is why Q21 prints undefined twice.

Figure A4.1 – *Work top to bottom: the first rule that applies wins. Rule 4 is the fallback.*

A4.2 Arrow functions break the rule on purpose

STORY BOX • THE CHILD WHO REPEATS WHAT THE PARENT SAYS

A small child is asked "who are you?" and does not answer for herself – she simply **repeats whatever her parent said**. Ask her in the kitchen and she gives the parent's answer; carry her to the park and she still gives the parent's answer. Her reply never depends on where *she* is standing, only on **whose child she is**.

An **arrow function** is that child. It has **no** `this` **of its own**. It looks outward to the function it was **written inside** and uses that one, permanently. Calling it differently cannot change the answer.

The technical name for "decided by where it was written" is **lexical** scoping. So: arrow functions take `this` **lexically** from the enclosing scope.

Regular function vs arrow function

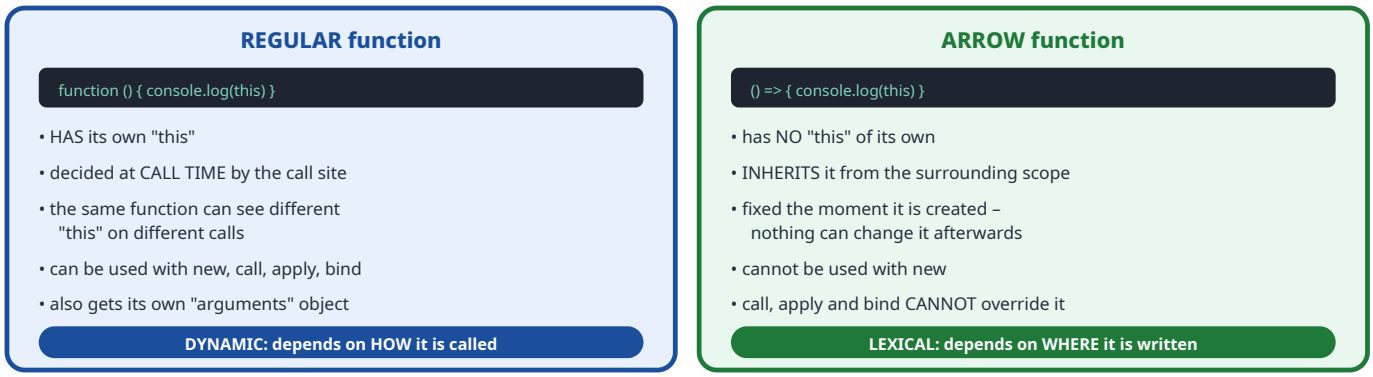

Figure A4.2 – *Arrow functions were added precisely so callbacks would stop losing* `this`.

Question 1

concept

Which of the following statements is true regarding the "this" keyword in JavaScript?

A. "this" in an arrow function refers to the global object.

B. "this" inside a regular function refers to the object that owns the function.

C. Arrow functions do not have their own "this" and inherit it from the surrounding scope.

D. "this" always points to the window object in browser environments.

Option-by-option reasoning

Option	Verdict	Why
A	FALSE	An arrow function inherits whatever `this` surrounds it. That is *sometimes* the global object – if the arrow sits at the top level – but it is the object when written inside a method. The word "refers to the global object" states one accidental case as if it were the rule.
B	FALSE	This is the option most students pick, because it is *almost* right and it is how the idea is often taught loosely. It fails because a function has **no permanent owner**. Take the very same function and call it as `fn()` and `this` is `window` or `undefined`, with no owner in sight. Ownership is not a property of the function; it is a property of **the call**. **Q21 exists specifically to disprove this option.**
C	TRUE the answer	Precisely correct, and it is the formal definition: arrow functions have no `this` binding of their own and resolve it **lexically** from the enclosing scope.
D	FALSE	The word **always** kills it. Inside a method call `this` is the object; in strict mode or a module it is `undefined`; in a constructor it is the new instance; in an event handler it is the element.

ANSWER TO QUESTION 1

Option C

Arrow functions do not have their own `this` and inherit it from the surrounding scope.

TRAP BOX • HOW TO CHOOSE BETWEEN B AND C UNDER PRESSURE

Both sound plausible. Use this test: **can I break the statement with one line of code?**

Option B breaks instantly – `const f = obj.method; f();` and the "owner" has vanished. Option C cannot be broken; even `arrowFn.call(otherObj)` fails to change an arrow's `this`. **The statement you cannot break is the true one.**

A4.3 Putting it together: the two-step method for any `this` puzzle

THE METHOD

1. **Is it an arrow function?** If yes, ignore the call entirely. Find the nearest enclosing *regular* function and work out **its** `this` instead.
2. **If it is a regular function, look at the call site.** Is there something immediately left of the dot? That is `this`. Is there nothing? Then `window` / `undefined`.

Question 21

JavaScript output

Consider the following code snippet running on a browser. What is the output on the console?

```
const obj = {
  value: 100,
  outerFunc: function () {
    const innerFunc = () => {        // ARROW
      console.log(this.value);
    };
    return innerFunc;
  },
  otherFunc: function () {         // REGULAR
    console.log(this.value);
  }
};

const func1 = obj.outerFunc();  // CALLED here, with the dot
const func2 = obj.outerFunc;    // only a reference, NOT called
const func3 = obj.otherFunc;    // only a reference, NOT called

func1();    // #1
func2()();  // #2
func3();    // #3
```

A. 100 100 100

B. 100 undefined 100

C. undefined undefined undefined

D. 100 undefined undefined

SPOT THE SETUP FIRST

Look carefully at the three `const` lines. **Only the first one has `()`**. So `func1` holds the *result* of calling `outerFunc`, while `func2` and `func3` hold the **functions themselves**, detached from `obj`. That single difference produces the whole answer.

Tracing all three calls

#1 func1() outerFunc was called as **obj.outerFunc()** – there IS a dot, so inside it "this" = obj
The arrow innerFunc was CREATED at that moment, so it captured "this" = obj, permanently.
Calling func1() later cannot change what the arrow captured.
prints obj.value = 100

#2 func2()() two calls, read them one at a time
First call, func2() – func2 is a bare variable. Nothing is left of a dot, so "this" is window / undefined.
The arrow is created NOW, inside that ownerless call, so it captures window – not obj.
Second call – runs the arrow, which reads window.value. No such property exists.
prints undefined

#3 func3() otherFunc is a REGULAR function, and it was detached from obj on the const line
Called as a bare func3() – no dot, no owner – so "this" is window / undefined.
window.value does not exist. **prints undefined**

Figure A4.3 – *The arrow in #1 is lucky and the arrow in #2 is unlucky, for the same reason:* ***what `this` was at the moment the arrow was created****.*

Verified output

```
100
undefined
undefined
```

ANSWER TO QUESTION 21

100 undefined undefined (option D)

Why each wrong option is tempting

Option	The misunderstanding it is fishing for
A	**100 100 100** – believing that a method "belongs to" its object forever, so `this` is `obj` whichever way you call it. This is exactly the belief that option B of Q1 describes, which is why the two questions belong together.
B	**100 undefined 100** – the sharpest distractor. You correctly saw that `func2()` loses `this`, but then assumed a plain regular method keeps its object. It does not: `func3` was detached on the `const` line too.
C	**all undefined** – over-correcting. You decided every detached call loses `this`, forgetting that `func1`'s arrow captured `obj` at creation time, while `outerFunc` was still being called *with* the dot.

THE SENTENCE THAT UNLOCKS THIS WHOLE FAMILY OF QUESTIONS

An arrow function freezes `this` at the moment it is created – so ask "what was `this` when the arrow was born?", not "how was the arrow called?"

In #1 the arrow was born inside `obj.outerFunc()`, so it froze `obj`. In #2 the arrow was born inside a bare `func2()`, so it froze `window`. Same arrow source code, two different frozen values.

EXTRA MARKS: HOW YOU WOULD FIX #2 AND #3

```
// force the owner explicitly
func3.call(obj);          // 100

// or permanently bind a copy
const bound = obj.otherFunc.bind(obj);
bound();                  // 100, however it is called

// or simply keep the dot
obj.otherFunc();          // 100
```

PART A5 • QUESTION 11

Classes, `extends` and the prototype chain

A5.1 Inheritance as a chain of "ask upstairs"

STORY BOX • ASKING YOUR WAY UP THE FAMILY

You need to know a recipe. You check your own notebook first. Not there, so you ask your mother. She does not know either, so she asks **her** mother. If the grandmother knows, you get the recipe – and you got it from the first person up the chain who had it.

JavaScript objects work identically. Ask `myDog.speak()` and the engine checks: does `myDog` itself have `speak`? No. Does **Dog** have it? Yes – stop there and use it. **Animal**'s version is never reached, because the search stops at the first match.

This search path is called the **prototype chain**, and "the first match wins" is what **method overriding** actually is. Nothing is deleted or replaced – the parent's method is still there, just further up the stairs.

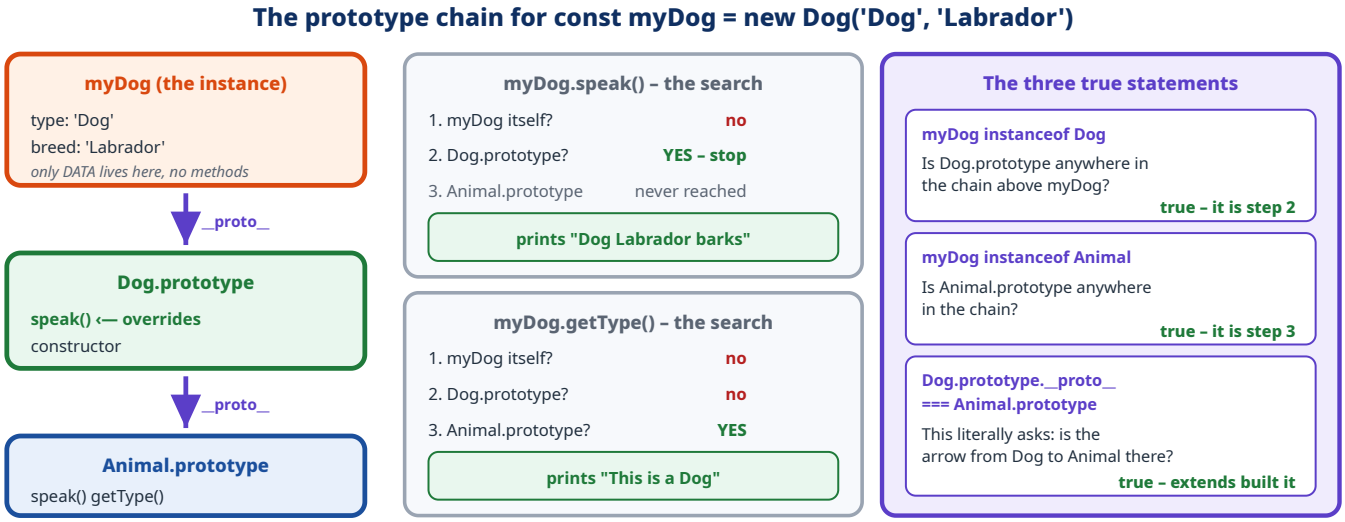

Figure A5.1 – *One diagram answers all five console lines.* `instanceof` *just walks upward looking for a prototype.*

DEFINITION BOX • THE VOCABULARY

- **`class Dog extends Animal`** – links `Dog.prototype` to `Animal.prototype`. That link is what `__proto__` names.
- **`super(type)`** – calls the parent constructor so the parent's setup (here `this.type = type`) actually happens. In a derived class you **must** call `super()` before touching `this`.
- **Overriding** – a child defines a method with the same name; the child's version is found first and wins.
- **`instanceof`** – true if the constructor's `prototype` appears **anywhere** in the object's chain, not just at the first level.
- **Methods live on the prototype, not on the instance.** A thousand dogs share one `speak` function.

Question 11

JavaScript output

What will be the output of the above program on a browser's console?

```
class Animal {
  constructor(type) { this.type = type; }
  speak()   { console.log(`${this.type} makes a sound`); }
  getType() { return `This is a ${this.type}`; }
}

class Dog extends Animal {
  constructor(type, breed) { super(type); this.breed = breed; }
  speak() { console.log(`${this.type} ${this.breed} barks`); }
}

const myDog = new Dog('Dog', 'Labrador');
myDog.speak();
console.log(myDog.getType());
console.log(myDog instanceof Dog);
console.log(myDog instanceof Animal);
console.log(Dog.prototype.__proto__ === Animal.prototype);
```

A. Dog Labrador barks / This is a Dog / true / true / true

B. Dog makes a sound / This is a Dog Labrador / true / false / true

C. Dog Labrador barks / This is a Dog Labrador / true / true / false

D. Dog Labrador barks / This is a Dog / false / true / true

Line-by-line solution

Line	Statement	Output	Reason
1	myDog.speak()	**Dog Labrador barks**	`Dog.prototype.speak` is found first and **overrides** the parent. `this.type` is `'Dog'` (set by `super`) and `this.breed` is `'Labrador'`.

2	myDog.getType()	**This is a Dog**	`Dog` has no `getType`, so the chain reaches `Animal.prototype.getType`. That method only ever mentions `this.type` – so **breed cannot appear**. This is the most-missed line.
3	myDog instanceof Dog	**true**	It was built with `new Dog(...)`.
4	myDog instanceof Animal	**true**	`instanceof` searches the **whole** chain, and `Animal.prototype` is one step further up. A Dog *is* an Animal.
5	Dog.prototype.__proto__ === Animal.prototype	**true**	This is exactly the link that `extends` creates. Asking it is asking "did inheritance happen?"

Verified output

```
Dog Labrador barks
This is a Dog
true
true
true
```

ANSWER TO QUESTION 11

Option A

Dog Labrador barks / This is a Dog / true / true / true

Why each wrong option is tempting

Option	The misunderstanding it is fishing for
B	Two errors. It runs the **parent's** `speak`, as if defining a method in the child does not override; and it says `instanceof Animal` is **false**, as if `instanceof` checked only the immediate class.
C	The sharpest distractor. Line 1 is right, and then `getType` is imagined to print **"This is a Dog Labrador"** – because you know the object *has* a breed and assume the inherited method will use it. **Read the parent method's body:** it only interpolates `this.type`. An inherited method sees all the data but only prints what it was written to print.
D	Says `myDog instanceof Dog` is **false**, which would mean `new Dog()` did not produce a Dog. Nonsense – but easy to tick if you are skimming the booleans.

TRAP BOX • THE TWO RULES TO SEPARATE

1. Overriding decides WHICH method runs. Child first, and the search stops there.

2. The method's own body decides WHAT it prints. Inheriting a method does not upgrade it – `getType` was written to mention only `type`, so that is all it will ever say, no matter how many extra properties the instance has.

PART A6 • QUESTION 2

Array methods and `reduce`

A6.1 Reduce is a running total

STORY BOX • THE SHOPKEEPER'S RUNNING TOTAL

A shopkeeper adds up your basket. He starts with **zero** on the paper. He picks up each item, reads its price, and writes the **new running total**. When the basket is empty, whatever is on the paper is the answer.

`reduce` is that process. The number on the paper is the **accumulator**, and the `0` at the end of the call is what he starts with. The function you pass says *"given the paper and the next item, what should the paper say now?"*

DEFINITION BOX • THE FOUR CALLBACK PARAMETERS

```
array.reduce((acc, val, idx, arr) => newAcc, initialValue)
```

acc	the **accumulator** – the running total so far
val	the **current element** being visited
idx	its index – available, but you do not have to use it
arr	the whole original array – rarely needed
initialValue	what the accumulator starts as. Here `0`.

Whatever you return becomes the next `acc`. Declaring extra parameters you never use changes nothing – that is the distraction in this question.

Every step of `[1, 2, 3].reduce((acc, val) => acc + val, 0)`

step	acc coming in	val	returns acc + val	acc for next step
start	–	–	initialValue	0
idx 0	0	1	0 + 1	1
idx 1	1	2	1 + 2	3
idx 2	3	3	3 + 3	6 FINAL

Array empty, so the accumulator 6 is the result. It is simply 0 + 1 + 2 + 3.

Figure A6.1 – *Write this four-row table on rough paper and any `reduce` question becomes arithmetic.*

Question 2

JavaScript output

What will be the output of the above program?

```
let arr = [1, 2, 3];
let sum = arr.reduce((acc, val, idx, array) => acc + val, 0);
console.log(sum);
```

A. 0

B. 3

C. 6

D. 9

E. 12

Solution

The callback declares four parameters, but the body uses only `acc + val`. The extra `idx` and `array` parameters are **received and ignored**, which changes nothing at all. So this is a plain sum starting from `0`: **0 + 1 + 2 + 3 = 6**.

Verified output

```
6
```

ANSWER TO QUESTION 2

6 (option C)

Why each wrong option is tempting

Option	What it corresponds to
0	Thinking the initial value is the result – i.e. that `reduce` never iterates, or that the arrow "returns nothing".
3	Two possible slips: returning only the **last** `val`, or confusing `reduce` with `arr.length`.
9	The trap built for the unused parameters: accidentally computing `acc + val + idx` gives 0+1+0, +2+1, +3+2 = 9. The examiner put `idx` in the signature purely to invite this.
12	Doubling the sum, e.g. by imagining the array is iterated twice or that `array` gets added in.

EXTRA MARKS: WHAT CHANGES IF THE INITIAL VALUE IS LEFT OUT

```
[1,2,3].reduce((a,b) => a+b)     // 6  – starts with acc=1, val=2
[].reduce((a,b) => a+b, 0)       // 0  – safe, initial value returned
[].reduce((a,b) => a+b)          // TypeError! empty array, no initial value
```

With no initial value, `reduce` uses the first element as the starting accumulator and begins at index 1 – and throws on an empty array. Always pass an initial value.

PART A7 • QUESTIONS 7 AND 14

Promises, async/await and error handling

A7.1 A promise settles exactly once, and then never changes

STORY BOX • THE EXAM RESULT ENVELOPE

A sealed envelope is prepared for you. While it is being prepared its state is **pending** – nobody knows the outcome yet.

Then the clerk stamps it. Either **PASS** or **FAIL**. Once that stamp is on, the envelope is **settled**. If the clerk immediately tries to stamp FAIL on top of a PASS, nothing happens – the envelope is already stamped and sealed. There is no error, no argument; the second stamp is simply **ignored**.

That is the single rule behind Q7. Calling `resolve()` and then `reject()` is not a mistake that crashes – the second call is silently discarded.

The promise state machine – note the one-way arrows

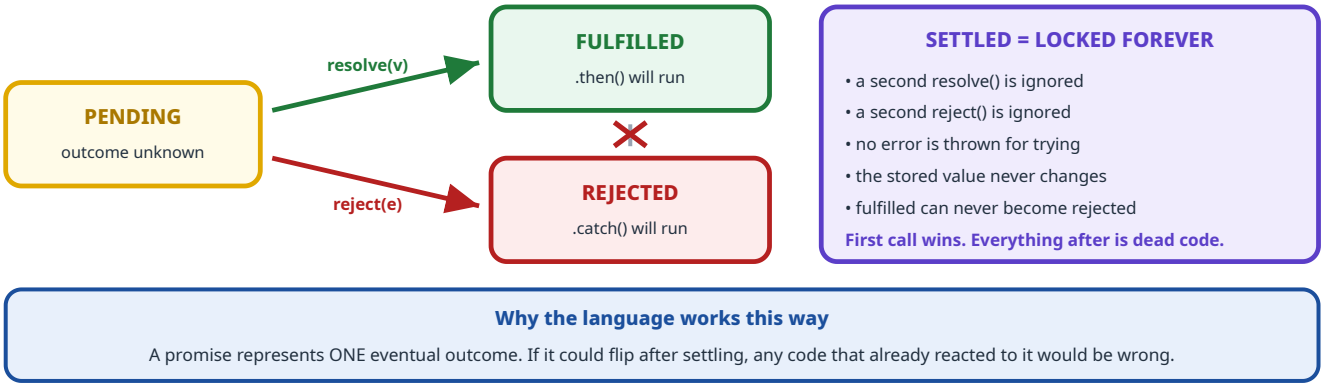

Figure A7.1 – *Once stamped, always stamped. There is no arrow back to pending and no arrow across.*

Question 7

JavaScript output

What will be the output of the above program?

```
const promise = new Promise((resolve, reject) => {
  resolve('Success!');     // runs FIRST -> the promise is now FULFILLED
  reject('Error!');        // too late. Silently ignored.
});
promise.then(console.log).catch(console.log);
```

A. Success!

B. Error!

C. Success! Error!

D. Error! Success!

Step-by-step solution

1 The executor runs immediately and synchronously

The function you pass to `new Promise` is not delayed. Both lines inside it run right away, in written order.

2 `resolve('Success!')` settles the promise

State goes `pending` › `fulfilled`, with the value `'Success!'`. The envelope is now stamped.

3 `reject('Error!')` is ignored

The promise is already settled, so this call does **nothing**. No error, no warning, no second output. It is effectively a dead line.

4 Only `.then` runs

The promise is fulfilled, so `.then(console.log)` prints `Success!`. Because nothing rejected and `.then` did not throw, the `.catch` is **skipped entirely**.

Verified output

```
Success!
```

ANSWER TO QUESTION 7

Success! (option A)

Why each wrong option is tempting

Option	The misunderstanding it is fishing for
B	**Error!** – thinking the **last** call wins, as if `reject` overwrites the earlier `resolve`. It cannot: settling is one-way.
C	**Success! Error!** – the most popular wrong answer. It assumes both handlers run because both functions were called. But `.then` and `.catch` react to the promise's **final state**, and a promise has exactly one.
D	Same mistake as C plus a guess about ordering.

TRAP BOX • WHEN DOES `.CATCH` ACTUALLY RUN?

Only in two situations: the promise **rejected**, or something in an earlier `.then` **threw**. A fulfilled promise with a clean `.then` skips `catch` completely. Writing a `reject` in the source code is not enough – it has to be the call that actually settles it.

A7.2 `async` / `await`: rejection becomes a thrown error

STORY BOX • THE TRANSLATOR

Promises speak one language: *"call my `.then` handler, or call my `.catch` handler."* Ordinary code speaks another: *"return a value, or throw an error."*

`await` is the **translator** between them:

- The promise **fulfils** › `await` hands you the value, like a normal return.
- The promise **rejects** › `await` **throws** that reason as an error, which a `try/catch` around it will catch.

This is why `async` code can use ordinary `try/catch` for asynchronous failures.

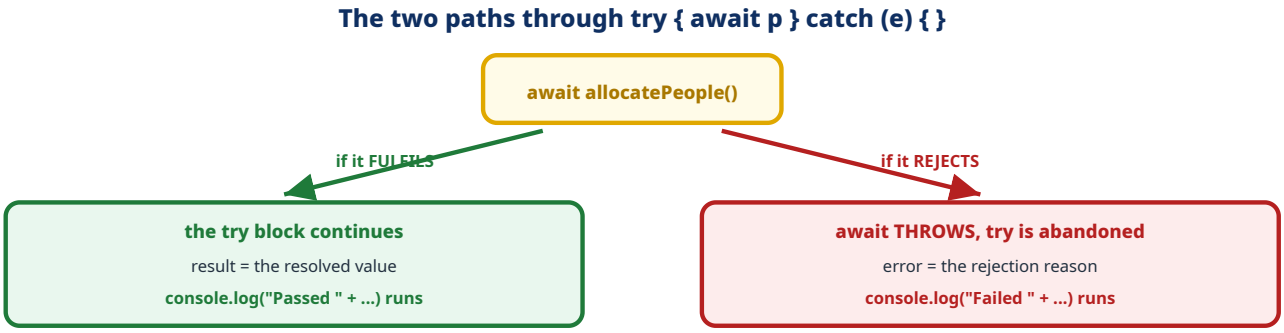

Figure A7.2 – *The rejection **reason** becomes the `catch` parameter. Whatever you passed to `reject()` is what you receive.*

THE DETAIL THAT DECIDES Q14

`reject(slots)` does **not** have to be given an `Error` object. You can reject with **anything** – a string, a number, or in this case an **array**. Whatever you pass arrives unchanged as the `catch` parameter. So `error` will be the array `[75, 75, 63]`, and `JSON.stringify` will print it as `[75,75,63]`.

Question 14

JavaScript output

What will be printed in the console when the above code is executed?

```
const CAPACITY = 75;
const TOTAL_PEOPLE = 213;

function allocatePeople() {
  return new Promise((resolve, reject) => {
    let slots = [0, 0, 0];
    let remaining = TOTAL_PEOPLE;

    for (let i = 0; i < slots.length; i++) {
      if (remaining >= CAPACITY) { slots[i] = CAPACITY; remaining -= CAPACITY; }
      else                       { slots[i] = remaining; remaining = 0; }
    }

    reject(slots);        // note: REJECT, not resolve
  });
}

async function startAllocation() {
  try {
    const result = await allocatePeople();
    console.log("Passed " + JSON.stringify(result));
  } catch (error) {
    console.log("Failed " + JSON.stringify(error));
  }
}
startAllocation();
```

A. Passed [75,75,63]

B. Failed [0,0,0]

C. Failed [75,75,63]

D. Passed [0,0,0]

TWO INDEPENDENT THINGS TO GET RIGHT

This question has **two** separate parts, and each wrong option gets exactly one of them wrong:

(1) WHAT is in `slots` ? – pure arithmetic in the loop.

(2) Which branch runs, Passed or Failed? – decided by `reject` .

Solve them separately, then combine.

Part 1 – trace the loop

i	remaining before	Is remaining >= 75?	slots becomes	remaining after
0	213	yes	[**75**, 0, 0]	213 - 75 = 138
1	138	yes	[75, **75**, 0]	138 - 75 = 63
2	63	**no** – 63 < 75	[75, 75, **63**]	0

Sanity check: 75 + 75 + 63 = **213**, exactly `TOTAL_PEOPLE`. Nobody is lost, so the arithmetic is right. Final `slots` is **`[75, 75, 63]`**.

Part 2 – which branch runs?

> **1** **The last line inside the promise is `reject(slots)`**
> Not `resolve`. There is no `resolve` call anywhere in this function. So the promise **always rejects**, no matter what the loop computed.

> **2** **`await` converts that rejection into a thrown error**
> The line `const result = await allocatePeople()` therefore **throws**. `result` is never assigned and the `"Passed"` line is never reached.

> **3** **Control jumps to `catch`, and `error` is the array**
> Because the rejection reason was `slots`, `error` *is* `[75, 75, 63]`. `JSON.stringify` renders it as `[75,75,63]`. Output: `"Failed " + "[75,75,63]"`.

Verified output

```
Failed [75,75,63]
```

ANSWER TO QUESTION 14

Failed [75,75,63] (option C)

Why each wrong option is tempting

Option	The misunderstanding it is fishing for
A	**Passed** [75,75,63] – you did the arithmetic perfectly and then **missed the word `reject`**. The examiner buried it on the last line, after a satisfying loop, hoping you would relax. This is the single most likely mistake on this question.
B	Failed **[0,0,0]** – you spotted the rejection but assumed the array was rejected in its *initial* state. But `reject` runs **after** the loop has already filled the array, and it passes a reference to that same, now-modified, array.
D	Both mistakes at once.

TRAP BOX • THE EXAM HABIT THAT SAVES THIS MARK

In any promise question, **find the settle call before you do any arithmetic**. Scan for `resolve` and `reject` first, circle whichever one actually executes, and write "PASS" or "FAIL" at the top of your rough work. Then compute the value. That way a long distracting loop cannot make you forget which branch you are in.

PART A8 • QUESTION 5

ES modules: `export` and `import`

A8.1 Named exports need their names back

STORY BOX • THE LABELLED PARCEL

A shop sends you a box containing **two labelled items**: one labelled *strikeRate* and one labelled *playerName*.

To take them out you must **ask for them by their labels**. You cannot just say "give me the box contents in order" – the shop never agreed on an order, only on labels. The curly braces in `import { strikeRate, playerName }` are you **reading out the labels**.

There is a second kind of parcel: one with a **single unlabelled main item** (a `default` export). That one you may name whatever you like, and you take it **without** braces. Mixing the two syntaxes up is what this question tests.

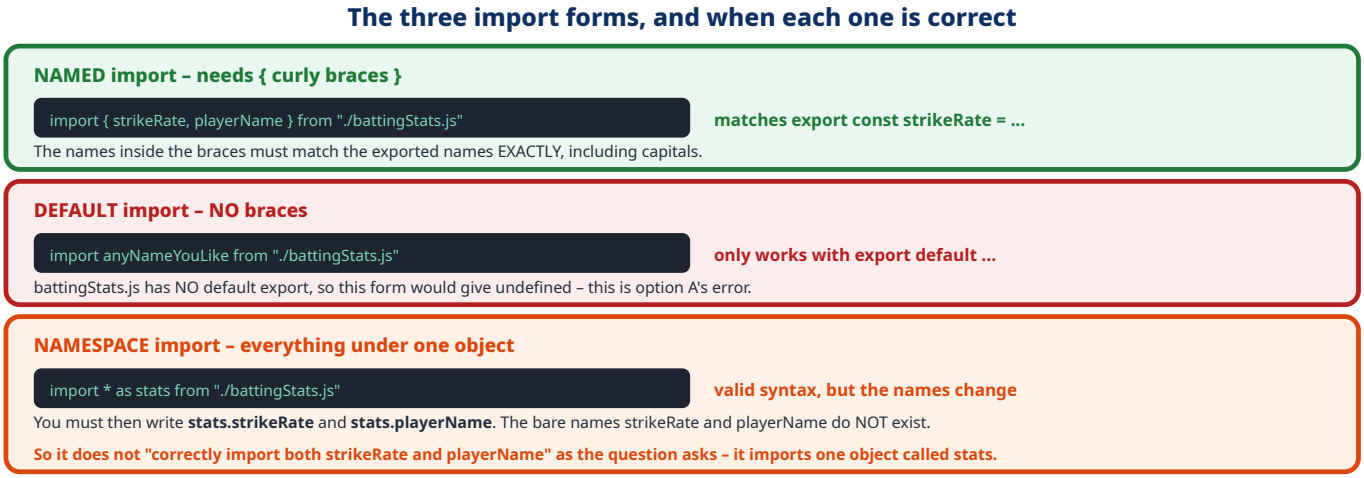

Figure A8.1 – *Braces or no braces is not style. It selects **which kind of export** you are asking for.*

Question 5

concept

The file `battingStats.js` contains:

```
export const strikeRate = (runs, balls) => {
  return (runs / balls) * 100;
};

export const playerName = "Virat Kohli";
```

In `app.js`, which import statement will correctly import both `strikeRate` and `playerName`?

A. `import strikeRate, playerName from "./battingStats.js";`

B. `import { strikeRate, playerName } from "./battingStats.js";`

C. `import * as stats from "./battingStats.js";`

D. `require("./battingStats.js");`

Option-by-option reasoning

Option	Verdict	Why
A	WRONG	No braces, so JavaScript reads `strikeRate` as a request for the **default** export – which this file does not have. The bare `, playerName` after it is not valid syntax either. This is a **SyntaxError**.
B	CORRECT	Braces request **named** exports, and the two names match the two `export const` declarations exactly. Both are then usable directly as `strikeRate(...)` and `playerName`.
C	WRONG for this question	This is **valid JavaScript** and it does load the module – which is why it is a good distractor. But it binds one object named `stats`. You would have to write `stats.strikeRate`. The question asks for an import of **`strikeRate` and `playerName`**, and after this line neither name exists on its own.
D	WRONG	Two problems. `require` is **CommonJS** (the older Node module system) and does not work with `export` / `import` ES module syntax in the same file. And even in CommonJS, this line imports the module but **assigns it to nothing**, so no names are introduced at all.

ANSWER TO QUESTION 5

Option B

`import { strikeRate, playerName } from "./battingStats.js";`

EXTRA MARKS: THE FULL EXPORT/IMPORT REFERENCE

In the exporting file	In the importing file
export const a = 1	import { a } from './m.js'
export function f() {}	import { f } from './m.js'
export default class X {}	import AnyName from './m.js'
export { a, b }	import { a, b } from './m.js'
export const a = 1	import { a as rate } from './m.js' (rename)
any exports	import * as ns from './m.js' (then ns.a)
default + named together	import Def, { a } from './m.js'

Note the one legal way to combine them, in the last row: the default comes first, bare, then the braces. That is why option A looks *almost* plausible – it has the shape of that form but with no braces at all.

PART B

The Vue Ecosystem

Twelve questions cover Vue itself, Vue Router and Vuex. The whole part rests on one idea: reactivity. Learn that and the rest is detail.

B0.1 The one idea underneath all of Vue

STORY BOX • THE SPREADSHEET

In a spreadsheet you type **10** into cell A1, and in cell B1 you write the formula `=A1*2`. B1 shows **20**.

Now change A1 to **15**. You do **not** go and edit B1. It updates itself, immediately, to 30. And a cell whose formula does not mention A1 does not recalculate at all – the spreadsheet knows exactly who depends on whom.

Vue is that spreadsheet, for a web page.

- `data` is the input cells (A1).
- `computed` is the formula cells (B1) – they recalculate only when something they depend on changes, and they **cache** otherwise.
- The **template** is the visible sheet – it redraws itself when the data behind it changes.

Every Vue question in this paper is asking: *"given this change, what recalculates, and when?"*

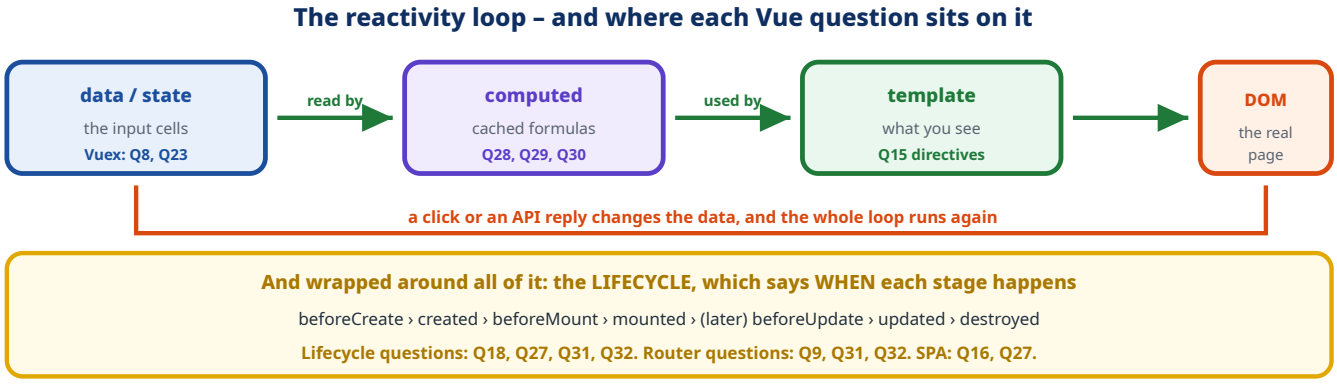

Figure B.0 – *Twelve questions, one diagram. Locate a question on this loop and you know what to reason about.*

PART B1 • QUESTION 15

Vue directives

STORY BOX • STICKY INSTRUCTIONS ON THE FURNITURE

You are arranging a room and you stick small notes on things:
• on a chair: *"only put this out if guests are coming"*
• on a stack of plates: *"lay one of these for each person"*
• on a picture frame: *"whatever photo is in the album, show it here"*
• on a whiteboard: *"if anyone writes here, update the album too"*

A **directive** is one of those notes, stuck onto an HTML element. It starts with `v-` and it tells Vue to do something to that element. The four notes above are exactly `v-if`, `v-for`, `v-bind` and `v-model`.

The four directives in this question, with real markup

v-if CONDITIONALLY RENDERS elements in the DOM

```
<p v-if="isLoggedIn">Welcome back</p>
```

If false the element is **not in the DOM at all** – not merely hidden.

v-for RENDERS A LIST of items

```
<li v-for="item in items" :key="item">{{ item }}</li>
```

One element per array entry. Always give a **:key**.

v-bind BINDS HTML ATTRIBUTES DYNAMICALLY

```
<img v-bind:src="photoUrl"> shorthand: :src="photoUrl"
```

The attribute follows the data. The **:** prefix IS v-bind.

v-model TWO-WAY DATA BINDING between input and data

```
<input v-model="username">
```

Data to input AND input to data. Used only on form fields.

Figure B1.1 – *Note the giveaway keywords: **conditionally**, **list**, **attributes**, **two-way**.*

Question 15

matching

Match the Vue directive with its correct functionality.

Directive	Description
1. v-if	A. Two-way data binding between input and data
2. v-for	B. Conditionally renders elements in the DOM
3. v-bind	C. Binds HTML attributes dynamically
4. v-model	D. Renders a list of items

A. 1-A, 2-D, 3-C, 4-B

B. 1-D, 2-B, 3-C, 4-A

C. 1-B, 2-C, 3-D, 4-A

D. 1-B, 2-D, 3-C, 4-A

Solution: match on the keyword

Directive	Keyword to look for	Match	Description matched
v-if	"**conditionally**"	**B**	Conditionally renders elements in the DOM – **if** is a condition, so this one is almost free.
v-for	"**list**"	**D**	Renders a list of items – **for** is a loop, and loops make lists.
v-bind	"**attributes**"	**C**	Binds HTML attributes dynamically – the word **bind** appears in both.
v-model	"**two-way**"	**A**	Two-way data binding – the **model** and the input stay in step with each other.

ANSWER TO QUESTION 15

1-B, 2-D, 3-C, 4-A (option D)

TRAP BOX • THE ONE CONFUSION TO NAIL DOWN

`v-bind` **is ONE-way;** `v-model` **is TWO-way.** Both contain the word "bind" in their description, and option C swaps them via a chain. On an input, `:value="x"` pushes data *into* the box but ignores typing; `v-model="x"` also brings typing back out. In fact `v-model` is just `v-bind` plus an event listener, bundled together.

Also worth knowing: `v-if` **vs** `v-show`. `v-if` removes the element from the DOM entirely; `v-show` keeps it and only sets `display: none`. Examiners like that pair too.

EXTRA MARKS: THE REST OF THE DIRECTIVE FAMILY

v-on:click / @click	attach an event listener
v-else / v-else-if	partner directives for `v-if`
v-show	toggle CSS visibility, element stays in the DOM
v-html	insert raw HTML – **an XSS risk**, use with care
v-text	set text content, same as `{{ }}`
v-once	render one time and never update again

PART B2 • QUESTION 18

The Vue lifecycle – and the trap inside it

This is the hardest Vue question in the paper, and it is hard for a reason most students never learn. Read this part carefully; it is worth several marks across any exam.

B2.1 The lifecycle is a birth sequence

STORY BOX • SETTING UP A NEW SHOP

Opening a shop happens in a fixed order, and **you cannot do a later step early**:

1. You sign the lease. The shop *exists* on paper, but it is an **empty room** – no shelves, no stock. **(beforeCreate)**
2. Stock arrives and goes on the shelves. Now you can count it and price it. **(created)**
3. You are about to unlock the front door and let customers see in. **(beforeMount)**
4. The door is open and customers can see the shop. **(mounted)**

Here is the crucial part. If at **step 1** you write "add 5 to the stock count", you are writing on a **stock register that does not exist yet**. When the real register arrives at step 2, it comes with its own printed numbers and your note is simply **gone**.

That is exactly what happens to `beforeCreate` in this question, and it is the whole answer.

DEFINITION BOX • THE ORDER, AND WHAT IS READY AT EACH POINT

Hook	Is `data` ready?	Is the DOM ready?	What it is for
1. beforeCreate	NO	NO	Almost nothing. Reactivity is not set up, so `this.someData` is `undefined`.
2. created	YES	NO	The usual place to **start an API call** or set up data. You can read and write `this.x` freely.
3. beforeMount	YES	NO	Last chance before the first render.
4. mounted	YES	YES	The DOM exists. Use it for anything needing real elements – measuring, charts, third-party widgets.

Memorise the order: beforeCreate › created › beforeMount › mounted. It reads oddly because "created" sits *between* the two "before" hooks – and that is precisely what the question exploits.

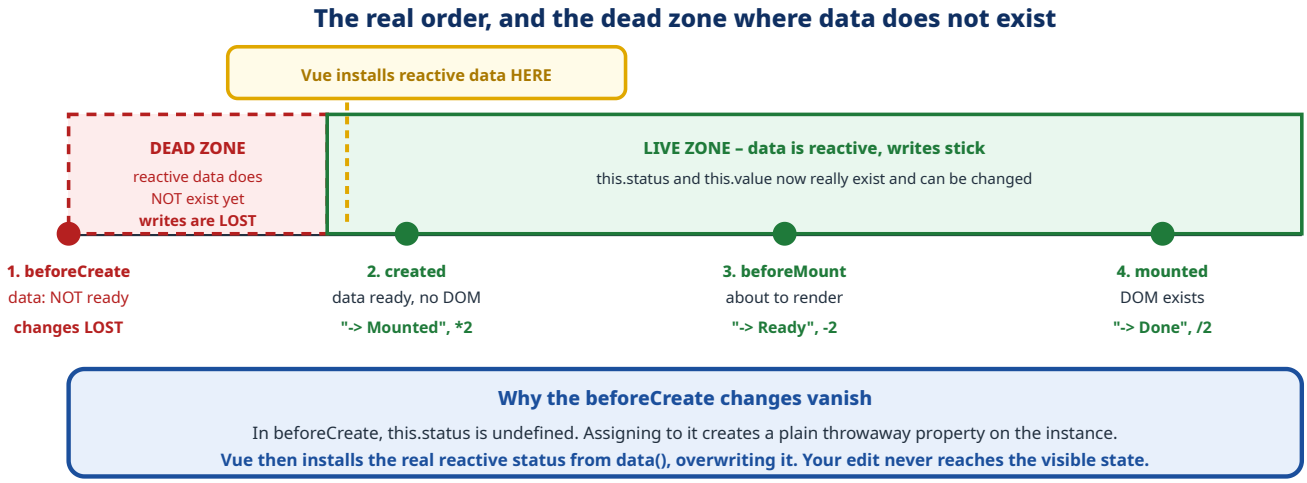

Figure B2.1 – *Two things must both be right: the **order** of the hooks, and the fact that hook 1 sits in a **dead zone**.*

Question 18

Vue output

Suppose the application is running on `http://127.0.0.1:8080`. What will be rendered by the browser?

```
new Vue({
  el: '#app',
  template: `<div>Status: {{status}}<br>Value: {{value}}</div>`,
  data: { status: "Initial", value: 10 },

  beforeCreate() { this.status = this.status + " -> Created"; this.value = this.value + 5; },
  beforeMount()  { this.status = this.status + " -> Ready";   this.value = this.value - 2; },
  created()      { this.status = this.status + " -> Mounted"; this.value = this.value * 2; },
  mounted()      { this.status = this.status + " -> Done";    this.value = this.value / 2; },
})
```

A. Status: Initial › Created › Ready › Mounted › Done Value: 13

B. Status: Initial › Ready › Mounted › Done Value: 8

C. Status: Initial › Created › Mounted › Ready › Done Value: 14

D. Status: Initial › Mounted › Ready › Done Value: 9

NOTICE THE DELIBERATE SCRAMBLING

The hooks are **written** in the order beforeCreate, beforeMount, created, mounted. That is **not** the order they run in. The order in the source file is irrelevant – Vue calls them in its own fixed sequence. The examiner shuffled them on purpose.

Step-by-step solution

1 Write down the real execution order

beforeCreate › created › beforeMount › mounted. So the operations actually happen in this sequence: `+5` (lost), `*2`, `-2`, `/2`.

2 Discard `beforeCreate`

At that moment `data` has not been installed, so `this.status` is `undefined` and `this.value` is `undefined`. The assignments create throwaway properties that Vue immediately overwrites when it sets up reactivity.

Net effect: nothing. Neither " -> Created" nor the +5 survives.

3 Now trace the three hooks that do count

Hook	status becomes	value becomes
start (from `data`)	"Initial"	10
beforeCreate	**skipped – data not ready, changes lost**	
created	"Initial -> Mounted"	10 * 2 = **20**
beforeMount	"Initial -> Mounted -> Ready"	20 - 2 = **18**
mounted	"Initial -> Mounted -> Ready -> Done"	18 / 2 = **9**

4 What finally appears on screen

`mounted` runs *after* the first render, but changing reactive data triggers a re-render, so the user sees the final values.

Status: Initial › Mounted › Ready › Done Value: 9

ANSWER TO QUESTION 18

Option D

Status: Initial › Mounted › Ready › Done, and Value: 9. Note the words look "out of order" precisely because the *hook named created* appends the text " -> Mounted".

Why each wrong option is tempting – this table is the real lesson

Option	Value	What belief produces it
C	14	**The expert's trap.** You know the correct hook order *and* you applied all four hooks: 10 +5 = 15, *2 = 30, -2 = 28, /2 = **14**. Everything right except the one fact that `beforeCreate` **cannot touch data**. If you chose this, you were one fact away – and it is the fact worth learning.
A	13	Assumes the hooks run in the order they are **written in the file**, and that all four apply. Neither is true.
B	8	Drops `beforeCreate` correctly but then runs the rest in **written** order: 10 -2 = 8, and the status string shows Ready before Mounted.

THE TWO FACTS TO CARRY AWAY

1. **Order:** beforeCreate › created › beforeMount › mounted – *not* the order written in the file.
2. **In `beforeCreate`, `data` does not exist yet**, so any change to `this.something` is silently thrown away. The first hook where data is usable is **`created`** – which is exactly why `created` is the standard place to start an API call (and why Q27 uses it).

PART B3 • QUESTIONS 28, 29 AND 30

Computed properties vs methods

B3.1 A computed property is a cached formula

STORY BOX • THE SHOPKEEPER WHO REMEMBERS THE TOTAL

A customer asks: *"what is the total of my basket?"* The shopkeeper adds it up – it takes a while – and writes the answer on a slip.

The customer asks again, twice more. The shopkeeper does **not** re-add anything; he just reads the slip. Three questions, **one calculation**.

Then the customer **adds a packet of biscuits** to the basket. Now the shopkeeper tears up the slip, because the basket changed. The next time he is asked, he adds up again.

That is a **computed property**: it calculates once, caches the answer, and only recalculates when one of the things it depends on has changed. A **method** is a shopkeeper with no slip – he re-adds the whole basket **every single time** he is asked.

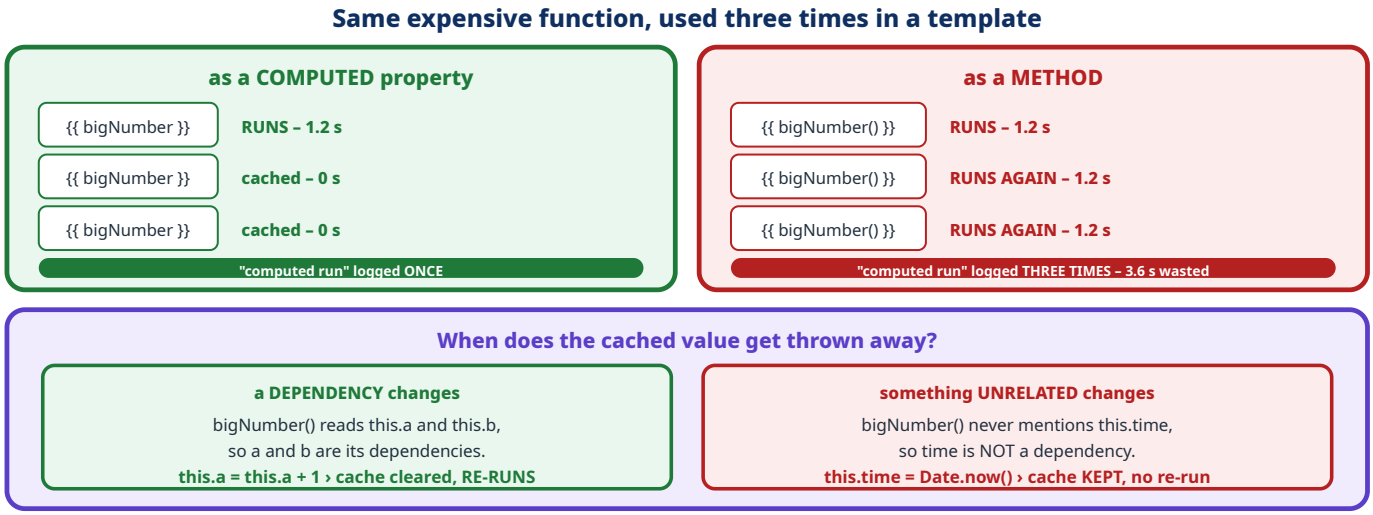

Figure B3.1 – *Vue tracks exactly which data a computed property read, and invalidates the cache only when **those** values change.*

DEFINITION BOX • COMPUTED VS METHOD

	computed	method
Used in template as	{{ bigNumber }}	{{ bigNumber() }}
Cached?	**yes**, until a dependency changes	**no**, runs on every render
Can take arguments?	no	yes
Right for	deriving a value from state	responding to an event, or work needing arguments
Must be pure?	yes – just calculate and return	no, may cause side effects

Question 28

Vue • subquestion 1

Given a `bigNumber` computed property that logs `"computed run"` and is used once as `{{ bigNumber }}` in the template – how many times will "computed run" be logged in the console during the initial render?

A. 0

B. 1

C. Multiple times due to the loop

D. Depends on user interaction

Solution

1 Is it evaluated at all?

Computed properties are **lazy** – they do not run until somebody reads them. The template does read it, via `{{ bigNumber }}`. So it runs. That rules out **0**.

2 How many times?

Once, and the result is cached. There is one render and one read, so exactly **one** evaluation.

3 What about the big loop inside it?

The `for` loop runs ten million times **inside** the function, but `console.log("computed run")` sits **outside** the loop, at the top of the function body. Running a loop inside a function does not make the function run repeatedly. That rules out **option C**, which is the trap.

ANSWER TO QUESTION 28

1 (option B)

TRAP BOX

Option C confuses **how many times the function is called** with **how much work happens inside one call**. Always check whether the `console.log` is inside or outside the loop. Here it is outside, so it prints once per call.

Question 29

Vue • subquestion 2

If the `calculate()` method is modified to `this.a = this.a + 1`, what will happen when the button is clicked?

- **A.** `bigNumber()` will not run again
- **B.** `bigNumber()` will run again because a dependency changed
- **C.** Only time will update
- **D.** Vue will throw an error

Solution

1 List the dependencies of `bigNumber`

Its body ends with `return s + this.a + this.b`. So it reads **`a`** **and** **`b`**. Those two are its dependencies. It never reads `time`.

2 What did the click change?

`this.a = this.a + 1` changes **`a`** – which *is* a dependency.

3 Therefore

The cached value is invalidated, the template re-renders, `bigNumber` is read again, and it **recalculates** – logging "computed run" a second time.

ANSWER TO QUESTION 29

Option B

`bigNumber()` will run again because a dependency changed.

THE CONTRAST THE EXAMINER IS BUILDING

Look at the **original** `calculate()`: it set `this.time = Date.now()`. Because `bigNumber` does **not** read `time`, clicking the button in the original version would **not** re-run the computed property – the cache would survive.

Change one line to touch `a` instead, and suddenly it does re-run. **Same button, opposite behaviour, decided purely by which data the computed function happens to read.** That is dependency tracking, and it is the point of both subquestions.

Question 30

Vue • subquestion 3

Why is `bigNumber()` defined as a computed property instead of a method?

A. Computed properties are always faster than methods

B. Methods cannot access data properties

C. Computed properties are cached based on their dependencies

D. Computed properties run only once

Option-by-option reasoning

Option	Verdict	Why
A	FALSE	The word **always** ruins it. For a single evaluation a computed property is not faster – it runs the identical code. The gain appears only on **repeat reads**, thanks to the cache. "Cached" is the precise reason; "faster" is a vague consequence.
B	FALSE	Plainly untrue – methods reach data through `this` all the time. The `calculate()` method in this very question writes `this.time`.
C	TRUE the answer	Exactly the defining feature, and exactly why it matters here: `bigNumber` contains a ten-million-iteration loop, so avoiding needless re-runs is the whole point.
D	FALSE	Tempting because in Q28 it *did* run once. But Q29 proves it runs again when a dependency changes. "Once" is wrong; "**once per change of its dependencies**" is right.

ANSWER TO QUESTION 30

Option C

Computed properties are cached based on their dependencies.

THE THREE SUBQUESTIONS AS ONE STORY

Q28: it runs **once**, because it is cached and lazily evaluated.
Q29: it runs **again**, but only because a dependency changed.
Q30: and **caching by dependency** is precisely the reason you would choose `computed` over a method in the first place.

All three are the same fact, asked from three angles.

PART B4 • QUESTIONS 9, 31 AND 32

Vue Router

B4.1 Routes are a list of patterns, checked in order

STORY BOX • THE RECEPTIONIST'S LIST

A receptionist has a list of instructions:

- *"Anyone asking for Accounts › send to room 1"*
- *"Anyone asking for Admissions › send to room 2"*
- *"**Anyone else at all** › send to Admissions"*

A visitor asks for "Gardening". The first two lines do not match, but the last line matches **everybody**, so off they go to Admissions.

That last catch-all line is the **wildcard route**, written `path: '*'` in Vue Router 2. Add a `redirect` and it becomes: *"anything unrecognised, send there instead"*.

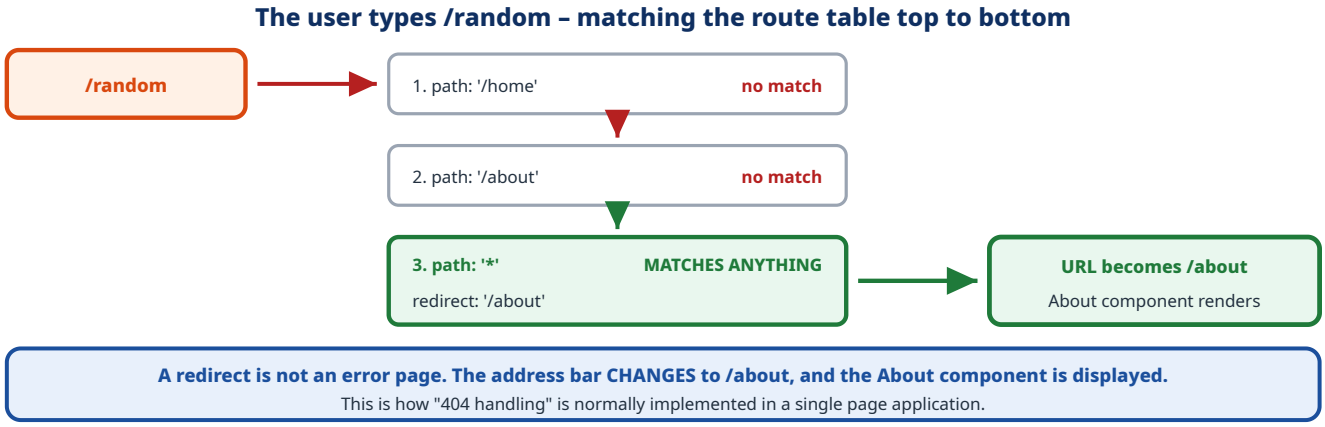

Figure B4.1 – *Order matters: a wildcard must be **last**, or it would swallow every route above it.*

Question 9

Vue Router

Given routes `/home`, `/about` and `{ path: '*', redirect: '/about' }` – what happens when the user navigates to an undefined route like `/random`?

A. Browser navigates to /random and shows a blank page.

B. Browser navigates to /random but renders nothing.

C. Browser redirects to /about and displays "About".

D. Error: No matching route.

Solution

`/random` matches neither `/home` nor `/about`, so matching continues to the third entry. `path: '*'` is the **wildcard** and matches every path, so it wins. Its `redirect: '/about'` then sends the router to `/about`, whose component is `About`, whose template is `<div>About</div>`. The URL updates and **"About"** is displayed.

ANSWER TO QUESTION 9

Option C

Browser redirects to /about and displays "About".

Why each wrong option is tempting

Option	The misunderstanding it is fishing for
A / B	Both describe what would happen if the wildcard route were **absent**: no route matches, `router-view` renders nothing, and you get a blank area with the URL unchanged. The whole point of the wildcard is to prevent this.
D	Vue Router does not throw a hard error for an unmatched path – at most it logs a warning. And here a route *did* match.

EXTRA MARKS: VERSION DIFFERENCE WORTH A MENTION

In **Vue Router 3** (used with Vue 2) the catch-all is `path: '*'`, as here. In **Vue Router 4** (Vue 3) that syntax was removed and you write `path: '/:pathMatch(.*)*'` instead. If a question shows `'*'`, you are in the Vue 2 world – which also tells you `new Vue({...})` and `this.$store` style code is expected.

B4.2 Reuse vs recreate: the idea behind Q31 and Q32

STORY BOX • CHANGING THE POSTER, NOT THE FRAME

A photo frame hangs on the wall showing picture 1. You swap in picture 2.

You did **not** take the frame off the wall, throw it away and hang a new one. You **reused the same frame** and changed its contents. The frame was never destroyed, so nothing about "a new frame being born" happened – but the frame's *appearance* definitely **updated**.

Vue Router does exactly this. Navigating from `/item/1` to `/item/2` keeps matching the **same route pattern** `/item/:id`, so the component is **reused**. Therefore:

- `created` and `mounted` do **not** run again – nothing was born.
- `updated` **does** run – the contents changed.

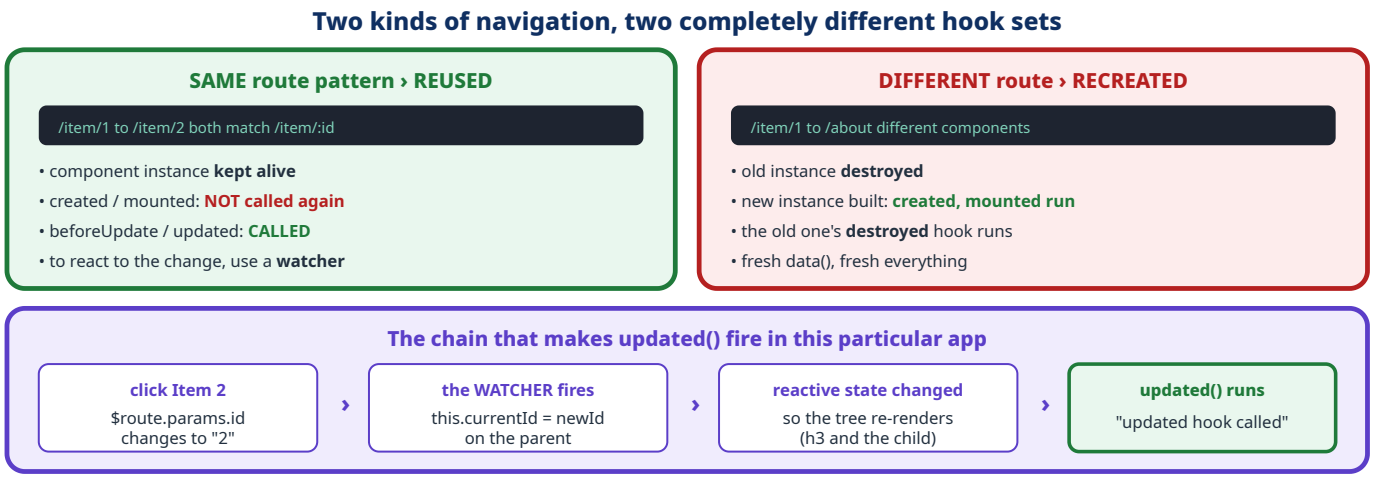

Figure B4.2 – *updated means "this component re-rendered", which is a very different event from "this component was created".*

Question 31

Vue Router • subquestion 1

The app is loaded at `/#/item/1` and the user clicks the Item 2 router link. What happens immediately after clicking "Item 2"?

A. The Item component is destroyed and recreated, so `updated()` is never called

B. The route changes, the component is reused, and `updated()` is called

C. The route changes, but neither watcher nor lifecycle hooks run

D. A new Vue instance is created for the route

Solution

1 Do both URLs match the same route?

There is exactly one route: `{ path: "/item/:id", component: Item }`. Both `/item/1` and `/item/2` match it – only the `:id` parameter differs. **Same pattern, so the component is reused.**

2 Does anything re-render?

Yes. The `Item` template interpolates `{{ $route.params.id }}`, which is reactive, and the parent's watcher also sets `currentId`. Both cause a re-render.

3 Which hook does a re-render trigger?

`beforeUpdate` and then `updated`. Since `Item` defines `updated()`, **"updated hook called" is logged**.

ANSWER TO QUESTION 31

Option B

The route changes, the component is reused, and `updated()` is called.

Why each wrong option is tempting

Option	The misunderstanding it is fishing for
A	The most common belief: "the URL changed, so surely the component is rebuilt". It would be true if you navigated to a **different route**. With the same `/item/:id` pattern Vue Router deliberately **reuses** the instance for efficiency – and that reuse is exactly why parameter changes are famous for catching people out.
C	Claims nothing runs. But a watcher was explicitly registered on `"$route.params.id"`, and the template depends on it, so both fire.
D	A new **Vue instance** is only created by `new Vue(...)`, once, at startup. Routing swaps components inside the existing instance; it never boots a second app.

Question 32

Vue Router • subquestion 2

Why is the `updated()` lifecycle hook triggered in this scenario?

A. Because Vue Router always recreates components on navigation

B. Because lifecycle hooks run on every URL change regardless of reactivity

C. Because router-view forces a full re-render of the component tree

D. Because a watched route parameter updates parent reactive state, triggering a component update

Option-by-option reasoning

Option	Verdict	Why
A	FALSE	Directly contradicts Q31. It does **not** always recreate – and if it did recreate, you would get `created` / `mounted`, **not** `updated`.
B	FALSE	"Regardless of reactivity" is precisely backwards. `updated` is a **consequence of reactivity**: it fires because reactive data changed and caused a re-render. A URL change that nothing depends on would render nothing new.
C	FALSE	`router-view` does not force anything. It renders whichever component matches, and Vue's ordinary dependency tracking decides what needs updating. Vue's whole design is to re-render as **little** as possible.
D	TRUE the answer	This traces the actual chain in this code: the **watcher** on `"$route.params.id"` fires, assigns `this.currentId` on the parent, that reactive change triggers a re-render of the tree, and the re-render is what invokes `updated()`. Figure B4.2.

ANSWER TO QUESTION 32

Option D

A watched route parameter updates parent reactive state, triggering a component update.

THE DISTINCTION WORTH MEMORISING

`created` / `mounted` answer *"was I born?"*. **`updated`** answers *"did I re-render?"*.

A reused component with changing parameters **re-renders without being reborn** – so it fires `updated` and never fires `created` again. This is the number one cause of a real-world Vue bug: fetching data in `created`, then wondering why the page does not refresh when only the URL parameter changes. **The fix is a watcher**, which is exactly what this app has.

PART B5 • QUESTIONS 8 AND 23

Vuex: state, mutations and actions

B5.1 Why a central store exists at all

STORY BOX • THE SCHOOL'S SINGLE REGISTER

A school could let every teacher keep their own private list of who is present. Then the office list, the sports teacher's list and the class list all disagree, and nobody knows which is real.

Instead the school keeps **one official register in the office**. Everyone **reads** from it. And crucially, only the **office clerk** is allowed to write in it – with a pen, in order, one entry at a time – so there is always a clear record of who changed what.

Vuex is that register. One shared `state` for the whole app, and a strict rule about who may change it.

DEFINITION BOX • THE FOUR PIECES, AND THE TWO VERBS

Piece	Job	Rules
state	the data	The single source of truth. Read it as `this.$store.state.x`.
mutations	change state	The **only** thing allowed to modify state. Must be **synchronous**. Triggered with `commit`.
actions	do async work	May wait, call APIs, use timers. Cannot change state directly – they `commit` a mutation when ready. Triggered with `dispatch`.
getters	derived values	Like computed properties, but for the store.

The two verbs are not interchangeable: `commit` runs a **mutation**; `dispatch` runs an **action**. Mixing them up is the trap in Q23.

The one-way Vuex flow – and where async work is allowed

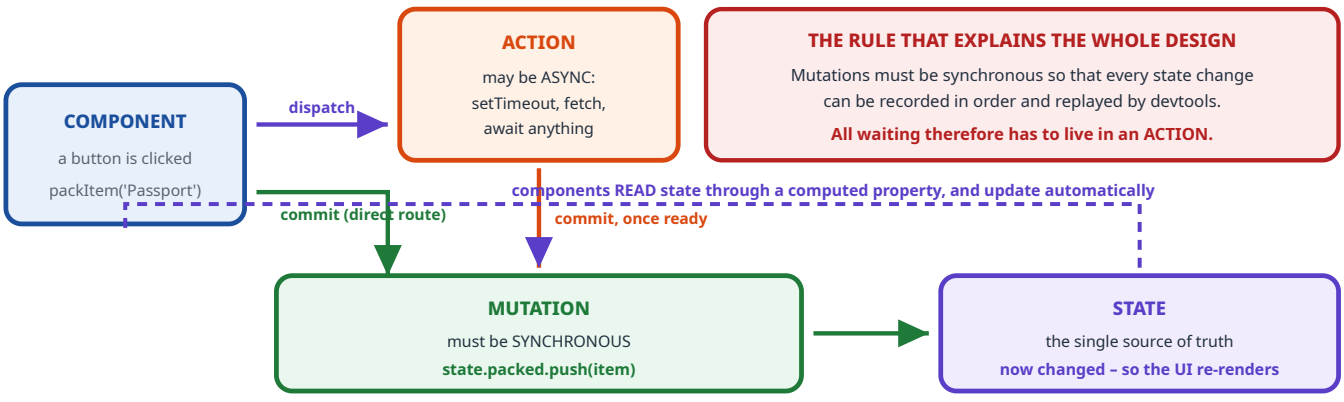

Figure B5.1 – *Actions wait; mutations write. Both routes end at the same mutation.*

Question 8

Vuex output

If the `addItemAsync` action is dispatched with the item 'React', what will be the state of `items` after 2 seconds?

```
const store = new Vuex.Store({
  state: { items: ['Angular', 'Vue'] },
  mutations: {
    addItem(state, item) { state.items.push(item); },
  },
  actions: {
    addItemAsync({ commit }, item) {
      setTimeout(() => { commit('addItem', item); }, 500);
    },
  },
});
```

A. ['Angular', 'Vue']

B. ['Angular', 'Vue', 'React']

C. ['React']

D. ['Angular']

Step-by-step solution

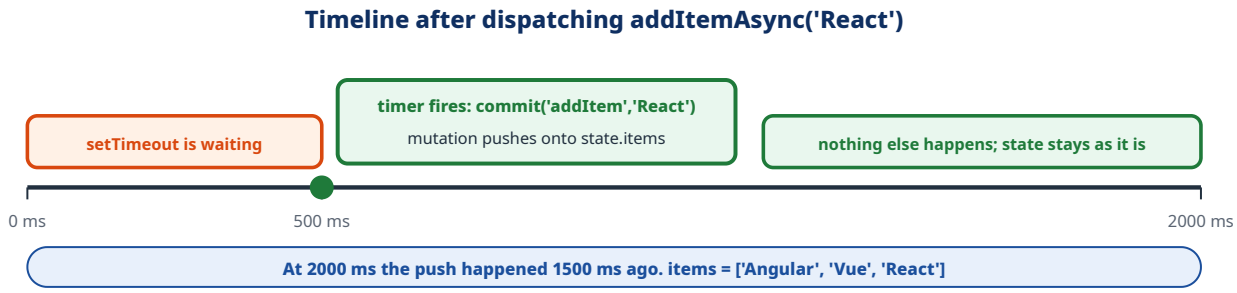

Figure B5.2 – *The only question is whether 2000 ms is longer than 500 ms. It is, by a wide margin.*

1 The action starts a 500 ms timer and returns immediately

That is legal, because **actions are allowed to be asynchronous**. Nothing has changed in the state yet.

2 At 500 ms the callback commits the mutation

`commit('addItem', 'React')` runs `state.items.push('React')`. Note it **pushes onto the existing array** – it does not replace it – so the two original entries stay.

3 We are asked about 2 seconds

2000 ms is well past 500 ms, so the push has definitely happened.

`['Angular', 'Vue', 'React']`

ANSWER TO QUESTION 8

['Angular', 'Vue', 'React'] (option B)

Why each wrong option is tempting

Option	The misunderstanding it is fishing for
A	The unchanged state. You would get this if you checked at, say, 200 ms – before the timer fired. The examiner chose **2 seconds** precisely so the timing is not close; misreading 500 ms as 5000 ms would produce this answer.
C	`['React']` – assumes the mutation **replaces** the array, as if it were `state.items = [item]`. But `push` appends.
D	Nothing in the code removes anything, so losing 'Vue' is impossible.

Question 23

Vuex • fill in the code

A Vuex store has `state: { packed: [] }` and a mutation `ADD_ITEM(state, item) { state.packed.push(item) }`. Two lines are missing: a computed property (Line X) that exposes the list, and a method body (Line Y) that adds an item. Which option fills both correctly?

A. `return this.store.state.packed;` / `this.store.commit("ADD_ITEM", item);`

B. `return this.$store.state.packed;` / `this.$store.commit("ADD_ITEM", item);`

C. `return this.$store.packed;` / `this.commit("ADD_ITEM", item);`

D. `packedItems: this.$store.state.packed` / `this.$store.dispatch("ADD_ITEM", item);`

The three things that must all be right

Requirement	Correct form	Why
1. The `$` prefix	this.$store	When you pass `store` to the root Vue instance, Vuex injects it into every component as **`this.$store`**. The `$` marks it as a framework-provided property. `this.store` is simply `undefined`.
2. The `.state` step	this.$store.state.packed	The store is a container. Its data lives under `.state`. You cannot skip that level and write `this.$store.packed`.
3. The right verb	this.$store.commit("ADD_ITEM", item)	`ADD_ITEM` is registered under **`mutations`**, and mutations are triggered with **`commit`**. `dispatch` looks for an **action** of that name – and there are no actions in this store at all.

Why each wrong option fails – each breaks exactly one rule

Option	Broken rule	Consequence
A	missing `$`	`this.store` is `undefined`, so `this.store.state` throws a TypeError immediately.
C	missing `.state`, and wrong receiver for commit	`this.$store.packed` is `undefined`, and `this.commit` does not exist on a component – `commit` lives on the store.
D	two errors	Line X is written as a **property assignment** rather than a computed **function**, so it would be evaluated once at definition time and never stay reactive. And `dispatch("ADD_ITEM")` looks for an action that does not exist, so nothing happens.

ANSWER TO QUESTION 23

Option B

Line X: `return this.$store.state.packed;` • Line Y: `this.$store.commit("ADD_ITEM", item);`

THE NAMING CONVENTION THAT GIVES YOU A FREE CLUE

Vuex projects conventionally write **mutations in SCREAMING_SNAKE_CASE** (`ADD_ITEM`) and **actions in camelCase** (`addItemAsync`).

Look at Q8 and Q23 side by side and you can see it: `addItemAsync` is an action and is `dispatch` ed; `ADD_ITEM` and `addItem` are mutations and are `commit` ted. So a name in capitals is a strong hint that `commit` is wanted.

PART B6 • QUESTIONS 16 AND 27

SPA routing and async data loading

B6.1 What a Single Page Application actually is

STORY BOX • REBUILDING THE ROOM VS CHANGING THE FURNITURE

The traditional way (server-side routing). Every time you want a different view, the builders **demolish the whole room** and construct a new one from scratch – walls, ceiling, lights, everything – even though only the poster on the wall needed to change. You stand outside on a blank site while they work. That is a **full page reload**.

The SPA way (client-side routing). The room is built once. To show you something new, someone walks in and **swaps the poster**. The walls, the lights and the ceiling are never touched. It is instant, and there is no moment of blankness.

The trade-off: the first build takes longer, because you had to ship the whole room-building kit (the JavaScript bundle) up front.

Server-side routing vs client-side routing

SERVER-SIDE (full reload)

Click a link, and the browser:

1. throws away the current page
2. asks the server for a whole new HTML page
3. re-downloads CSS and JS (or reads cache)
4. rebuilds the entire DOM from zero
5. loses all in-memory JavaScript state

You see a white flash between pages.
Every navigation costs a server round trip.

Upside: excellent SEO by default, because the server sends finished HTML.

CLIENT-SIDE (SPA)

Click a link, and JavaScript:

1. intercepts the click, no page unload
2. swaps the component inside router-view
3. maybe fetches a small JSON payload
4. keeps the shell, sidebar and header intact
5. keeps all in-memory state

Seamless transitions, no full reloads.
The server sends data, not whole pages – so less load.

Downside: SEO is WORSE by default, because the first response is a nearly empty page.

Figure B6.1 – *Note the SEO row: it is the opposite of what students usually assume, and it is the trap in Q16.*

Question 16

select all that apply

In a Single Page Application, what are the benefits of client-side routing over server-side routing?

- **A.** Slower navigation between pages
- **B.** Reduced load on the backend server
- **C.** Better SEO performance by default
- **D.** Seamless page transitions without full reloads

Option-by-option reasoning

Option	Verdict	Why
A	FALSE	Not merely wrong – it is not even a **benefit**. Read the question stem: it asks for benefits. "Slower" could never qualify, so you can eliminate this without knowing anything about SPAs. Navigation is in fact **faster** after the first load.
B	TRUE	The server stops rendering and shipping full HTML pages for every click. It serves the bundle once, then small JSON responses. Less template rendering, less bandwidth, fewer requests – a genuine reduction in backend load.
C	FALSE	**Backwards, and the real trap.** A SPA's first response is often just `<div id="app"></div>`; the content appears only after JavaScript runs. That makes SEO **harder**, not easier – which is exactly why server-side rendering (Nuxt, Next) and pre-rendering exist. Note the phrase **"by default"**: SEO can be fixed with extra work, but it is not a default benefit.
D	TRUE	The defining feature. The shell stays mounted, only the routed component is swapped, so there is no white flash and no loss of state.

ANSWER TO QUESTION 16

B and D

Reduced load on the backend server, and seamless page transitions without full reloads.

TRAP BOX • TWO FREE ELIMINATIONS ON ANY "BENEFITS" QUESTION

1. Any option that names a downside cannot be a benefit. Option A describes slowness – gone immediately.
2. Watch for "by default". It converts a "possible with effort" claim into a "happens automatically" claim, and that is usually what makes the option false.

B6.2 Why a component with an API call looks slow

STORY BOX • THE MENU AND THE FOOD

You sit down in a restaurant. The **menu, the table and the cutlery** are already there – they were in the room before you arrived, so you see them instantly.

You order a dish. The **plate arrives several minutes later**, because it had to be cooked after you asked.

Nothing is broken. The furniture was **pre-existing**; the food was **fetched on demand**. In the Vue app, the **sidebar is the furniture** (part of a parent that already rendered) and the **assignments list is the food** (fetched by an API call started in `created()`).

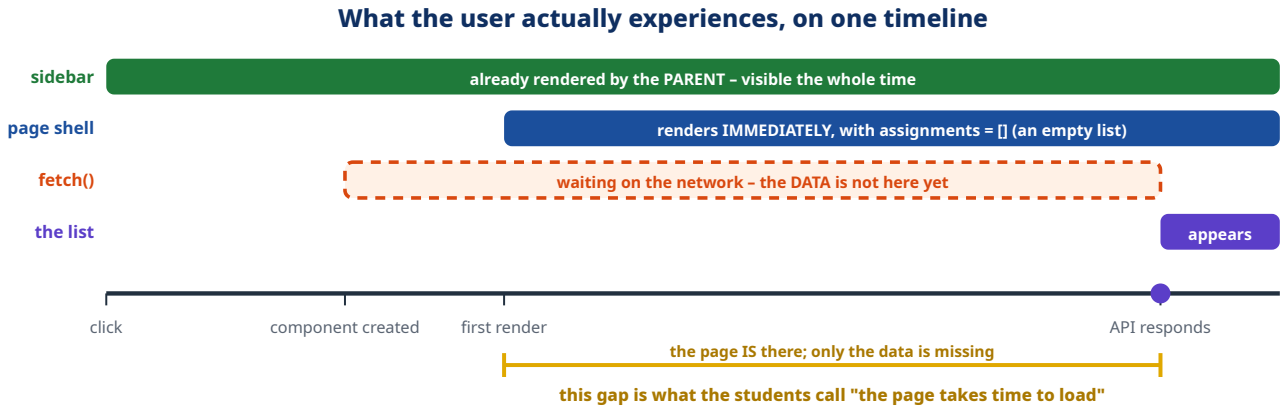

Figure B6.2 – *The component renders at once with an empty array. The **data** is late, not the page.*

Question 27

select all that apply

The sidebar appears instantly, but clicking Assignments takes noticeable time. The component is:

```
export default {
  data() { return { assignments: [] } },
  created() {
    fetch("/api/assignments")
      .then(res => res.json())
      .then(data => { this.assignments = data; });
  }
}
```

Which statements correctly explain why the Assignments page takes time to load?

A. The sidebar is likely part of a parent component and is already rendered before the API call

B. The Assignments component waits for the API response before displaying data

C. Vue blocks rendering of all components until API calls complete

D. The delay is caused because data is fetched asynchronously after the component is created

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	This explains the **contrast** the question opens with. The sidebar lives in a parent or layout component that mounted when the app started, and SPA navigation never unmounts it. So it has nothing to wait for – it was already on screen.
B	TRUE	Read it precisely: the component waits for the response before **displaying data**. That is exactly right. The markup renders immediately, but with `assignments = []` there is nothing to show until the response lands and reactivity re-renders the list.
C	FALSE	**The trap.** Vue never blocks rendering on network calls – and the question's own premise disproves it. If Vue blocked *all* components, the **sidebar would also have stalled**, and we are told it appeared instantly. Rendering is synchronous and immediate; the fetch resolves later and triggers a second render.
D	TRUE	The root cause, stated precisely. The fetch is **asynchronous** and is started in `created()` – so the component's existence does not depend on it, and the data arrives strictly afterwards.

ANSWER TO QUESTION 27

A, B and D

Only C is false, and the question's own wording contradicts it.

USE THE PREMISE AGAINST THE OPTIONS

The stem tells you a fact for free: **"the sidebar content appears instantly"**. Any option claiming that Vue blocks *all* rendering must therefore be false, because the sidebar rendered. You can eliminate C from the question stem alone, without reading the code.

EXTRA MARKS: HOW YOU WOULD FIX THE PERCEIVED SLOWNESS

- Show a **loading skeleton or spinner** while `assignments` is empty – the wait becomes visible and intentional rather than looking broken.
- Keep a `loading: true` flag in `data` and clear it in the final `.then`.
- **Cache** the response so a second visit is instant.
- Move the fetch into a **route guard** if you would rather not navigate until the data has arrived.
- Make the API itself faster – often the real answer.

PART C

Flask and the Backend

Seven questions on the server side: request threading, caching, JWT authentication, role-based authorisation and GraphQL.

PART C1 • QUESTION 22

Flask request threading and timing

STORY BOX • ONE WAITER OR MANY WAITERS

A restaurant gets two orders at the same moment: a dosa that takes 10 minutes and a biryani that takes 30.

With many waiters, each order gets its own waiter and both start cooking at once. The dosa is ready at minute 10, the biryani at minute 30.

With one waiter who refuses to touch a second table until the first is completely served, the biryani cannot even *start* until minute 10 – so it arrives at minute 40.

Flask's development server is **Restaurant A by default**. Calling `app.run()` quietly sets `threaded=True`, so requests are handled **simultaneously**. You only get the one-waiter behaviour if you write `threaded=False` yourself.

THE FACT TO MEMORISE, AND THE DIRECTION PEOPLE GET WRONG

Since **Flask 1.0**, `app.run()` defaults to **`threaded=True`**. Many students assume the opposite – that Flask is single-threaded until you ask for threads. Remember the direction: **threaded is the default; `threaded=False` is a downgrade you choose.**

Underneath, Flask hands your settings to the Werkzeug development server, whose *own* default is `threaded=False` – which is exactly why the fact is worth memorising rather than guessing.

C1.1 The two lines that carry all the information

```
@app.route('/')
def home():
    time.sleep(10)                    # 1. FIRST freeze for 10 s
    return jsonify(datetime.now().second), 200    # 2. THEN read the clock
```

DEFINITION BOX • THE THREE PIECES

- `time.sleep(10)` – the function stops dead for 10 seconds. It stands in for genuinely slow work: a big query, a report, another company's API.
- `datetime.now().second` – the **seconds part only**, a number from 0 to 59. The hours and minutes are discarded, so 10:11:05 gives `5`, not 65.
- The clock is read **after** the sleep. So the number you get is **(time the function started) + (sleep duration)**, expressed as seconds only.

The `jsonify`, the `200` and the CORS header change nothing about timing.

C1.2 Both fetches leave the browser in the same instant

In `index.html`, `fetch()` **sends the request immediately** and hands back a promise. JavaScript does not wait, so line 2 and line 3 both fire at 10:10:10. The `.then` chains only say *what to do when a reply arrives*; they do not delay anything. And the console prints in **order of arrival**, not order of code.

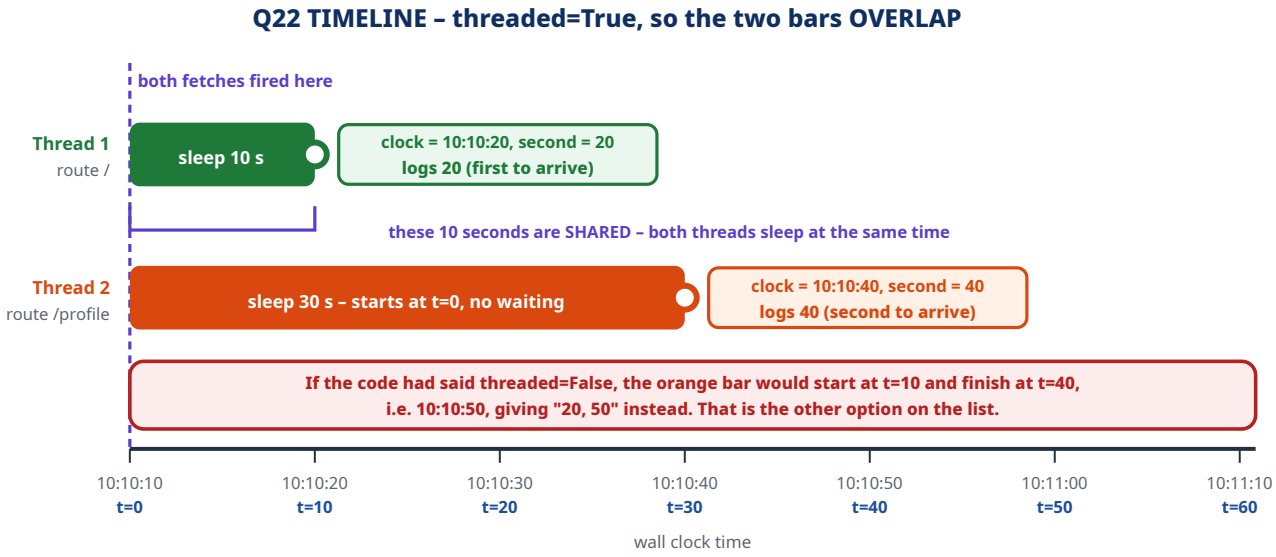

Figure C1.1 – *Both bars start at t = 0 because each request gets its own thread.*

Question 22

Flask timing

With `app.run(debug=True)`, a user visits `index.html` at 10:10:10 AM. What will be logged in the console?

A. 20, 50

B. 20, 40

C. 40, 40

D. 50, 50

Step-by-step solution

1 Check the server mode
The code says `app.run(debug=True)` – no `threaded` argument, so Flask's default `threaded=True` applies. **Many waiters.**

2 Set t = 0 at 10:10:10
Both fetches leave at that instant.

3 Request 1 to `/`
Its own thread sleeps 10 s, reaching the return line at t = 10 = **10:10:20**. Seconds = **20**.

4 Request 2 to `/profile`
A *separate* thread, also starting at t = 0 with no waiting. Sleeps 30 s, reaching the return line at t = 30 = **10:10:40**. Seconds = **40**.

5 Printing order
Reply 1 lands at t = 10, reply 2 at t = 30, so the console prints **20** then **40**.

ANSWER TO QUESTION 22

20, 40 (option B)

Why each wrong option is tempting

Option	The misunderstanding it is fishing for
A	**20, 50** – assumes `/profile` had to queue behind `/` (10 + 30 = 40 s of waiting, landing at 10:10:50). True only with `threaded=False`. **This is the paired question examiners love to set right after this one** – if you see `threaded=False` in the code, the answer becomes 20, 50.
C	**40, 40** – assumes both routes finish together. Impossible: one sleeps 10 s and the other 30 s.
D	Both mistakes at once: queueing *and* equal finish times.

THE REUSABLE FORMULA FOR EVERY QUESTION OF THIS TYPE

answer second = (start second + time spent waiting + own sleep) mod 60

- **Threaded** server: time spent waiting = **0**.
- **Single-threaded**: it is the total sleep of everything ahead of you in the queue.
- `mod 60` matters: starting at 10:10:45 with a 30 s sleep gives 75 mod 60 = **15**, not 75.

PART C2 • QUESTION 13

Flask-Caching and `memoize`

C2.1 A cache is a labelled shelf

STORY BOX • THE LABELLED TIFFIN BOXES

Amma cooks slowly – five minutes a dish – and stores each finished dish in a box on a shelf. To find a dish again she needs a **label** on the box.

If every box is labelled just **"FOOD"**, then the first dish takes the shelf and every later request gets handed that same box. Ask for rajma and you receive the poha that got there first.

If the labels instead read **"FOOD - user 1"** and **"FOOD - user 2"**, everything works.

That label is the **cache key**, and the entire difference between Flask-Caching's two decorators is **what goes on the label**.

DEFINITION BOX • `CACHED` VS `MEMOIZE`

- `@cache.cached()` builds its key from the **request path** (default `view/<request.path>`). The function's **arguments are not part of the key**.
- `@cache.memoize()` builds its key from the **function plus its arguments**. Different arguments therefore get **different cache entries**.

Anchor it with the word: **memoization** means, by definition, caching a function's result **per set of arguments**. Arguments are baked into the meaning of the word.

Three requests, and what the cache does with each

request	cache key it looks up	hit or miss?	work done	time taken
user 1	compute_data + (1)	**MISS – empty shelf**	runs, sleeps 5 s	**5 seconds**
user 2	compute_data + (2)	**MISS** different key!	runs, sleeps 5 s	**5 seconds**
user 1 again	compute_data + (1)	**HIT** stored 5 s ago	function NOT run no sleep at all	**about 0 seconds**
TOTAL = 5 + 5 + 0 = 10 seconds				

Why the third request is free: memoize put the ARGUMENT in the key, so user 1's result was stored separately
and the timeout of 300 s has not expired – the whole scenario happens inside 5 minutes.

Figure C2.1 – *Two misses and one hit. The hit is what turns 15 seconds into 10.*

Question 13

Flask-Caching

If requests for user IDs 1, 2, 1 are made within 5 minutes, what is the total response time?

```
app.config['CACHE_TYPE'] = 'SimpleCache'
app.config['CACHE_DEFAULT_TIMEOUT'] = 300
cache = Cache(app)

@cache.memoize(timeout=300)
def compute_data(user_id):
    time.sleep(5)
    return f"Data for user {user_id}"

@app.route('/api/info/<int:user_id>')
def get_info(user_id):
    return compute_data(user_id)
```

A. 15 seconds

B. 10 seconds

C. 5 seconds

D. 0 seconds

Step-by-step solution

1 Which decorator is used?

`@cache.memoize` . So the cache key includes the **argument** `user_id` . User 1 and user 2 get **separate** cache entries.

2 Is the timeout long enough?

`timeout=300` is 300 seconds = 5 minutes, and all three requests happen **within** 5 minutes. So nothing expires before the third request.

3 Count the misses

Request for user 1: **miss**, so the function runs and sleeps 5 s.
Request for user 2: **miss** (different key), runs, sleeps 5 s.
Request for user 1 again: **hit**, the function is not called, so no sleep at all.

4 Add it up

5 + 5 + 0 = **10 seconds**.

ANSWER TO QUESTION 13

10 seconds (option B)

Why each wrong option is tempting

Option	The misunderstanding it is fishing for
A	**15 s** – three requests times five seconds, i.e. **ignoring the cache entirely**. This is the answer if you did not notice the decorator, or if you assumed a cache needs configuring further to take effect.
C	**5 s** – the opposite error, and a very instructive one: it assumes the **second** request also hits the cache. That would be true with **`@cache.cached`**, which ignores arguments and would happily serve user 1's data to user 2. With `memoize`, user 2 is a genuine miss.
D	**0 s** – assumes the cache is pre-warmed. The first call to any function must always miss; a cache cannot contain a value nobody has computed.

TRAP BOX • THE BUG THAT `@CACHED` WOULD HAVE CAUSED HERE

Suppose the decorator had been `@cache.cached(timeout=300)` on this parameterised function:

```
@cache.cached(timeout=300)
def compute_data(user_id): ...

compute_data(1)  # miss -> "Data for user 1" stored under a path-based key
compute_data(2)  # HIT on the SAME key -> returns "Data for user 1"  WRONG
```

Total time would be **5 seconds** – and user 2 would see user 1's data. That is not just a performance question, it is a **data-leak bug**. Use `memoize` for any function whose result depends on its arguments.

PART C3 • QUESTIONS 3, 6 AND 25

JWT and Flask-JWT-Extended

C3.1 The problem authentication has to solve

STORY BOX • THE CLOAKROOM TICKET VS THE WRISTBAND

You arrive at an event and prove who you are once, at the door. How does every stall inside know you are allowed in?

Method 1 – the cloakroom ticket (sessions). The door staff write your details in a big **ledger** and hand you a numbered ticket. Every stall must radio the office and ask *"is ticket 4172 valid, and who is it?"* The office must keep that ledger for every visitor, all day. If you open a second gate, it needs the same ledger too.

Method 2 – the tamper-proof wristband (JWT). The door staff write your name and expiry **on the band itself** and seal it with a hologram only they can produce. Now any stall can read the band and check the hologram **on the spot**. **No ledger, no phone call, no memory.**

That is the meaning of **stateless**: the token carries its own proof, so the server does not have to remember anything about you.

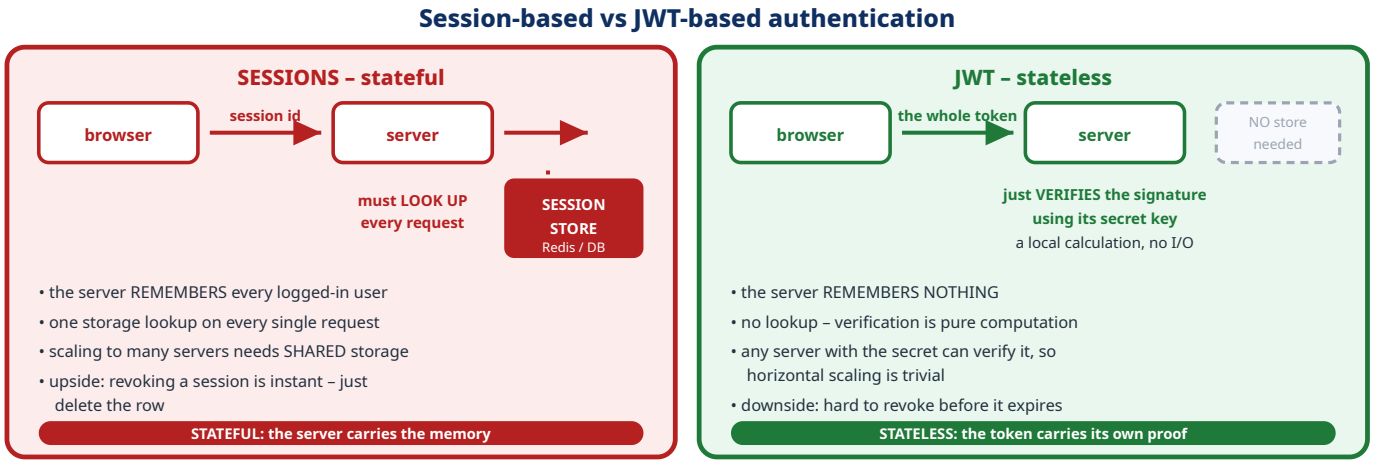

Figure C3.1 – *The single biggest architectural difference: whether the **server** has to store anything per user.*

Question 3

concept

What is the main advantage of using JWT over session-based authentication?

A. JWTs require server-side storage for each session.

B. JWTs are stateless and eliminate the need for server-side session storage.

C. JWTs are longer and contain more data than session IDs.

D. JWTs automatically handle token expiration without additional logic.

Option-by-option reasoning

Option	Verdict	Why
A	FALSE	The exact opposite of the truth, and it describes **sessions**. JWT's whole selling point is needing **no** per-user server storage.
B	TRUE the answer	Statelessness is the headline advantage, and it is what makes JWT popular for APIs and microservices: any server holding the secret can verify a token without shared storage.
C	FALSE	**True as a statement, but it is a disadvantage, not an advantage.** A JWT really is much longer than a session id, so it costs more bytes on every request. Read the question: it asks for the main *advantage*. A factually correct sentence can still be the wrong answer.
D	FALSE	Half true, and the sharpest distractor. A JWT *can carry* an `exp` claim, and libraries *do* reject expired tokens. But "automatically … without additional logic" overstates it: **you** must set the expiry when creating the token, and handling **refresh** is entirely extra logic you write yourself. Compare with B, which is unconditionally true.

ANSWER TO QUESTION 3

Option B

JWTs are stateless and eliminate the need for server-side session storage.

TRAP BOX • "TRUE" IS NOT THE SAME AS "THE ANSWER"

Option C is a genuinely true fact about JWTs. It fails only because the question asked for an **advantage**. Always re-read what the stem is asking for – advantage, disadvantage, cause, purpose – before comparing options.

C3.2 What is actually inside a JWT

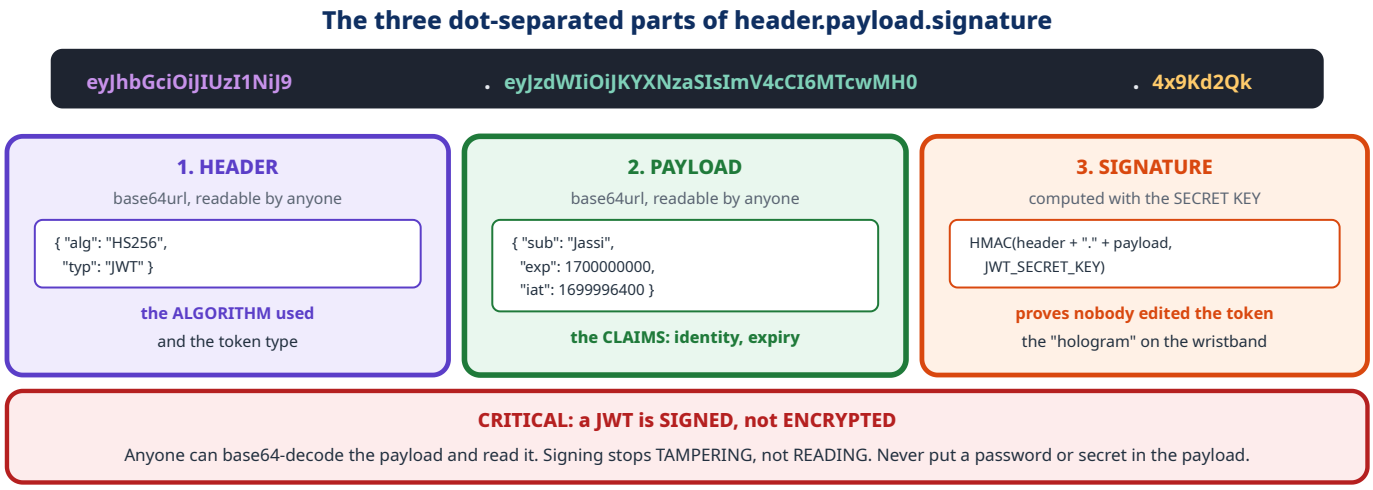

Figure C3.2 – *Header says how it was signed, payload says who and until when, signature proves it was not altered.*

Question 6

concept

A JWT is represented as `"header.payload.signature"`. In authentication using Flask-JWT-Extended, what does the payload part store?

- **A.** Encryption keys used to secure the token
- **B.** User-related data (claims) such as identity and expiration time
- **C.** The algorithm used for signing the token
- **D.** The hashed signature of the token

Option-by-option reasoning

Option	Verdict	Why
A	FALSE	Dangerously wrong. The payload is **publicly readable**, so putting a key in it would hand your secret to every client. The secret key lives **only on the server** (`app.config["JWT_SECRET_KEY"]`) and never travels.
B	TRUE the answer	Exactly right, and the word **claims** is the technical term. In Flask-JWT-Extended, `create_access_token(identity=username)` puts the identity into the payload as the `sub` claim, and the library adds `exp` (expiry) and `iat` (issued at) automatically. `get_jwt_identity()` later reads it back out.
C	FALSE	The algorithm is in the **header** (`"alg": "HS256"`), which is part 1, not part 2. This option is describing the right token – but the wrong third of it.
D	FALSE	The signature is **part 3**. The format string in the question literally lists the parts in order, so the payload cannot also be the signature.

ANSWER TO QUESTION 6

Option B

User-related data (claims) such as identity and expiration time.

FREE MARKS: THE QUESTION HANDS YOU THE ANSWER

The stem spells out `header.payload.signature` **in order**. So options C ("the algorithm" = header) and D ("the signature" = signature) are each describing a *different named part*. Two options eliminated by reading the question, before you know any JWT theory at all.

C3.3 How the token travels: the Authorization header

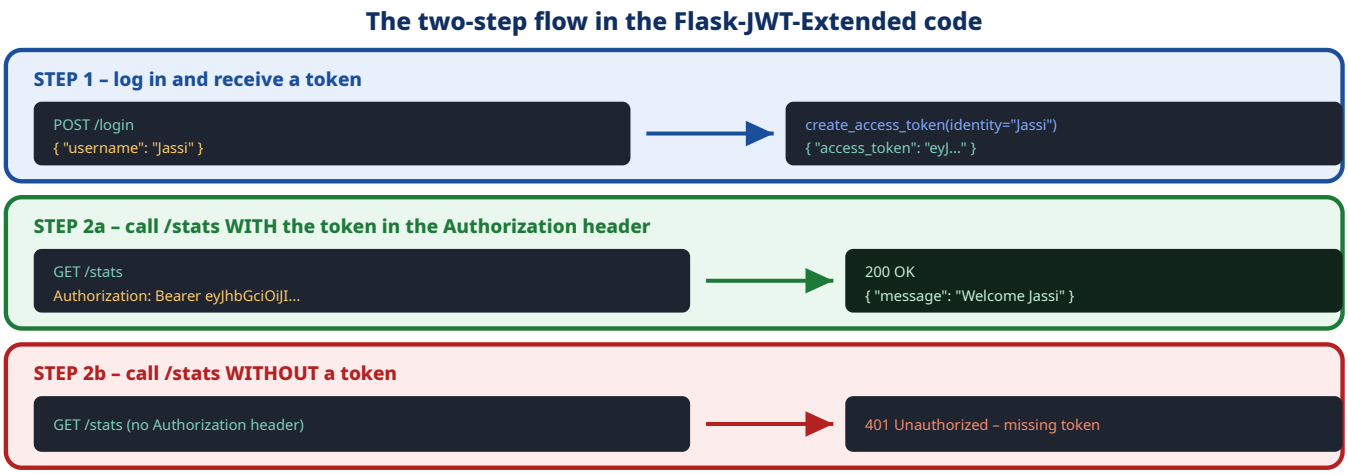

Figure C3.3 – `@jwt_required()` *looks in the* ***Authorization header****, in the form* `Bearer <token>` *– never in the request body.*

Question 25

select all that apply

A user POSTs to `/login` with `{ "username": "Jassi" }`, then tries to access `/stats`. Which statements are correct?

A. The `/stats` route will return "Welcome Jassi" only if a valid JWT token is sent in the Authorization header

B. If no token is sent, accessing `/stats` will result in an unauthorized error

C. The JWT token must be sent as JSON in the request body to access `/stats`

D. None of these

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	Two things must both hold, and this option states both. The route is decorated with `@jwt_required()`, so a **valid** token is needed; and `get_jwt_identity()` returns whatever was passed as `identity` at login – here `"Jassi"` – producing `"Welcome Jassi"`. The word **"only if"** is correct too, since without a valid token the view never runs.
B	TRUE	With no token, Flask-JWT-Extended rejects the request before your function is reached, returning **401 Unauthorized**. Note the option says "an unauthorized error" rather than naming a code, which keeps it safely true.
C	FALSE	Wrong location. By default the token is read from the **`Authorization: Bearer <token>`** header. A header is not the body. (For completeness: Flask-JWT-Extended *can* be configured to accept tokens from cookies or query strings via `JWT_TOKEN_LOCATION` – but this code does not configure that, and "must be sent as JSON in the request body" is wrong regardless.)
D	FALSE	Fails because A and B are both true.

ANSWER TO QUESTION 25

A and B

EXTRA MARKS: THINGS WORTH NOTICING ABOUT THIS CODE

- **There is no password check at all.** `login()` reads a username and issues a token to anybody who asks. Fine for a teaching example, a catastrophe in production.
- `JWT_SECRET_KEY = "secret"` is a placeholder. A real key must be long, random and kept out of source control.
- `@jwt_required()` needs the **brackets** in Flask-JWT-Extended 4.x. Older 3.x code wrote `@jwt_required` without them – a classic version gotcha.
- The scheme word is **`Bearer`**, and it is case-sensitive by convention.

PART C4 • QUESTION 19

Flask-Security, roles, and 401 vs 403

C4.1 Authentication and authorisation are two different gates

STORY BOX • THE OFFICE BUILDING

There are **two** checks between the street and the server room, and they fail in different ways.

Gate 1 – the reception desk: "who are you?" You show your ID card. No card, or a fake card, and you are stopped here. The guard does not know who you are, so he cannot even begin to consider whether you are allowed in. That is **authentication**, and failing it gives **401 Unauthorized**.

Gate 2 – the server room door: "are YOU allowed in HERE?" Your card is genuine and the guard knows exactly who you are – you are Ravi from accounts. But the server room is for IT staff only. **Recognised, and still refused.** That is **authorisation**, and failing it gives **403 Forbidden**.

The distinction in one line: **401 means "I do not know you". 403 means "I know you, and the answer is no."**

DEFINITION BOX • THE TWO DECORATORS IN THE CODE

- `@auth_required('token')` – gate 1. Is there a valid authentication token? Missing or invalid gives **401**.
- `@roles_required('superuser')` – gate 2. Does this identified user hold that role? Wrong role gives **403**.
- **No decorator** – no gate at all. Anyone gets **200**.

Note the order in the code: `@auth_required` is listed **above** `@roles_required`, so identity is always checked first. You cannot be refused for the wrong role until the server knows who you are.

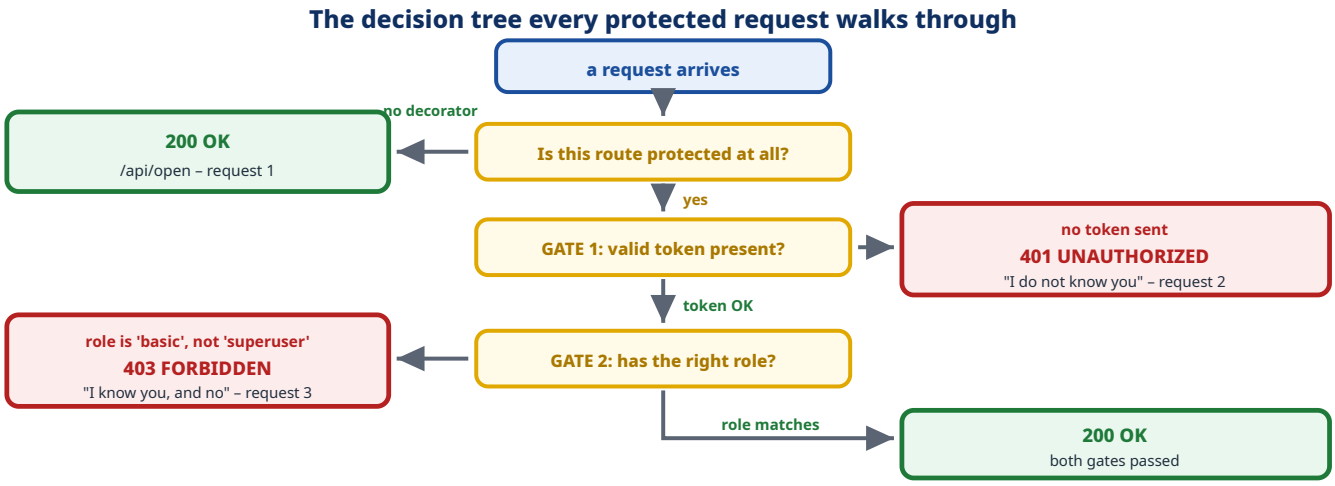

Figure C4.1 – *The three requests in the question take three different paths down this one tree.*

Question 19

Flask-Security

Three requests are made: (1) GET `/api/open` with no authentication headers; (2) GET `/api/secure` with no authentication headers; (3) GET `/api/superuser` with a valid token but the user has role `'basic'`, not `'superuser'`. What are the response status codes, respectively?

```
@app.route('/api/open')
def open_endpoint(): ...              # no protection

@app.route('/api/secure')
@auth_required('token')               # needs identity
def secure_endpoint(): ...

@app.route('/api/superuser')
@auth_required('token')               # needs identity
@roles_required('superuser')          # AND needs the role
def superuser_endpoint(): ...
```

A. 200, 200, 200

B. 200, 401, 403

C. 200, 403, 401

D. 401, 403, 403

Request-by-request solution

#	Endpoint	What is sent	Code	Reason
1	/api/open	nothing	**200**	No decorator, so no gate. It is deliberately public and returns its JSON happily.
2	/api/secure	no auth headers	**401**	`@auth_required('token')` finds no token. The server cannot identify the caller, so **Unauthorized**. Note it stops at gate 1 – there is no role question to ask yet.
3	/api/superuser	valid token, role 'basic'	**403**	Gate 1 **passes** – the token is valid and the user is known. Gate 2 fails, because `'basic'` is not `'superuser'`. Identified but not permitted, so **Forbidden**.

ANSWER TO QUESTION 19

200, 401, 403 (option B)

Why each wrong option is tempting

Option	The misunderstanding it is fishing for
A	**200, 200, 200** – assumes the decorators are documentation rather than enforcement. They genuinely block the request before your view function runs.
C	**200, 403, 401** – the codes are simply **swapped**, and this is the option most people fall for. It is worth having a permanent rule to prevent it, which is in the box below.
D	**401, 403, 403** – claims even the unprotected route needs auth. Nothing in the code protects `/api/open` .

THE PERMANENT RULE FOR 401 VS 403

401 = "who are you?" unanswered. No credentials, bad credentials, expired token. The correct fix is **to log in**.
403 = "you, specifically, may not." Credentials were fine; permission was not. Logging in again will **not** help – you need a different role.

A memory hook for the confusing names: the word "Unauthorized" in 401 is a historical misnomer – it really means **unauthenticated**. Read 401 as *"un-authenticated"* and 403 as *"un-authorised"*, and they stop swapping.

PART C5 • QUESTION 24

GraphQL

C5.1 Ordering exactly what you want

STORY BOX • THE FIXED THALI VS THE A-LA-CARTE ORDER

A REST API is a fixed thali. You ask for "the lunch plate" and you get whatever is on it – rice, two vegetables, curd, pickle. If you only wanted the rice, you still receive and pay for all of it (**over-fetching**). If you also wanted a sweet, that is a second, separate order (**under-fetching**, needing more round trips).

GraphQL is ordering a-la-carte, in one sentence. "Rice, and the sweet, and nothing else, please." One request, exactly the fields you named, nothing wasted.

And the kitchen is described by a **menu with strict definitions**: what dishes exist, what each one contains, and who prepares it. That menu is the **schema**, and the "who prepares it" part is the **resolver** – one function per field.

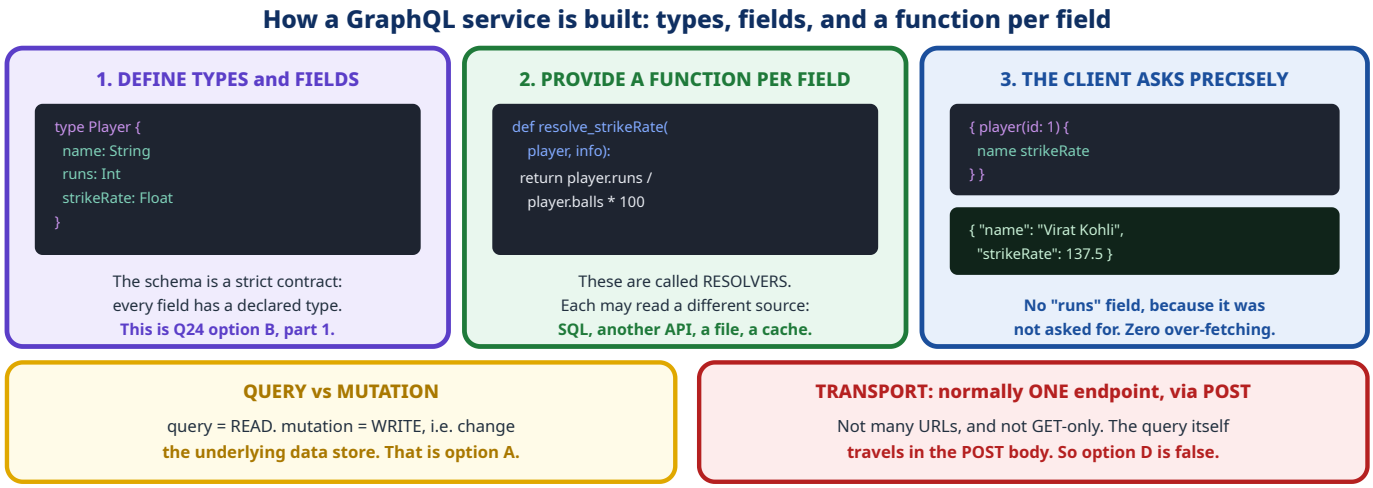

Figure C5.1 – *Because each field has its own resolver, different fields can come from **completely different sources**.*

Question 24

select all that apply

Which of the following statement(s) is/are true about GraphQL?

A. Mutations in GraphQL can be used to change the underlying data store.

B. GraphQL services are created by defining types and fields of those types, then providing functions for each field.

C. GraphQL can only fetch data from a single source.

D. GraphQL allows only GET operations.

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	GraphQL has two main operation types. A **query** reads; a **mutation** writes – creating, updating or deleting data in the underlying store. That is precisely what mutations exist for.
B	TRUE	This is almost word for word how the official documentation describes building a GraphQL service: **define types and fields**, then **write a function for each field** that returns its value. Those functions are the resolvers.
C	FALSE	The word **only** gives it away, and the reality is the opposite: because every field has its own resolver, a single query can pull `name` from a SQL database, `weather` from a third-party API and `avatar` from a file store, then return them as one JSON object. **Aggregating multiple sources is one of GraphQL's main selling points.**
D	FALSE	Wrong twice. GraphQL is not limited to reading – mutations exist, as option A says, so the option even contradicts A. And in practice GraphQL is served over a **single endpoint using POST**, with the query in the request body.

ANSWER TO QUESTION 24

A and B

INTERNAL CONSISTENCY: A FREE CHECK ON MULTI-SELECT QUESTIONS

Options A and D **contradict each other**. A says mutations change data; D says only GET operations are allowed, and GET is for reading. They cannot both be true. So the moment you are confident about A, you know D is false without any further knowledge – and vice versa. Look for these contradictions; they turn four decisions into two.

EXTRA MARKS: THE GRAPHQL VOCABULARY EXAMINERS USE

Schema	the typed contract: what types and fields exist
Query	a read operation
Mutation	a write operation – create, update, delete
Subscription	a live stream of updates, usually over WebSockets
Resolver	the function that produces one field's value
Over-fetching	REST giving you fields you did not need
Under-fetching	needing several REST calls to assemble one screen
Single endpoint	typically `/graphql`, rather than many URLs

PART D

Platform and Tooling

Three questions on things every web developer is expected to know: the WebSocket handshake, cookies and performance, and the Git workflow.

PART D1 • QUESTION 4

The WebSocket handshake

STORY BOX • ASKING TO SWITCH FROM LETTERS TO A PHONE CALL

You have been writing letters to an office – one letter, one reply, each time a fresh envelope. That is HTTP.

Now you want a **live phone call** instead, so either side can speak at any moment. But you have no phone line yet. So how do you ask? **You send one last letter** saying: *"please switch us to a phone call"*. The office writes back *"agreed"*, and from that moment the letters stop and the call is live.

That final letter is the **WebSocket handshake**, and it is an ordinary **HTTP GET** request. It has to be, because at that point HTTP is the only channel available.

Figure D1.1 – *HTTP is used once, to negotiate its own replacement.*

Question 4

concept

Which of the following HTTP methods does WebSocket use for the initial handshake?

- **A.** GET
- **B.** POST
- **C.** PUT
- **D.** DELETE

Solution, and how to reason it out if you have forgotten

1 The handshake request carries no data

It is asking permission to change protocol, not submitting anything. Its whole content is in the **headers**: `Upgrade: websocket` and `Connection: Upgrade`. A request with no body is naturally a **GET**.

2 Eliminate the others by meaning

POST means "here is data to process", **PUT** means "store this at this URL", **DELETE** means "remove this resource". None of those describes "please upgrade this connection".

3 The specification agrees

The WebSocket protocol requires the opening handshake to be a **GET** request, answered with **101 Switching Protocols**.

ANSWER TO QUESTION 4

GET (option A)

EXTRA MARKS: FACTS OFTEN ASKED ALONGSIDE THIS

Handshake method	GET
Success status code	101 Switching Protocols
Key headers	Upgrade: websocket • Connection: Upgrade • Sec-WebSocket-Key
Schemes	ws:// and secure wss://
Direction	full duplex – both sides send freely
Contrast with SSE	SSE stays pure HTTP and is **one way** (server to client) only
Contrast with a webhook	a webhook is a single POST that closes immediately – nothing stays open

PART D2 • QUESTION 17

Cookies and performance

D2.1 A cookie is attached to every single request

STORY BOX • THE ID CARD YOU MUST SHOW EVERY TIME

A building makes you show your ID at the door **every time** you walk in – and you enter 80 times a day. If the card is a thin plastic chip, fine. If it is a fat **4 KB folder of documents**, you have just added that weight to **all 80 entries**.

A cookie is that folder. The browser attaches it automatically to **every** request for the domain – every image, every stylesheet, every icon – whether that request needs it or not. So cookie size is **multiplied by the number of requests**. That is why it affects performance.

Worse, this weight is on the **upload** direction, which on most home and mobile connections is far slower than download.

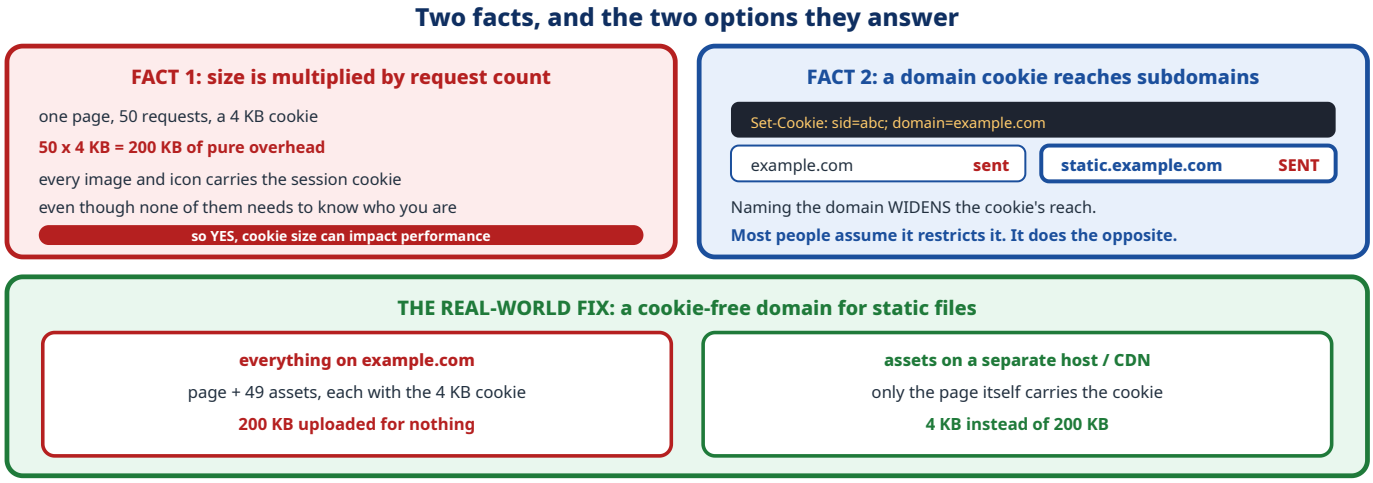

Figure D2.1 – *Cookies are for identity. A logo does not need to know who you are, so sending your session with it is wasted bytes.*

Question 17

select all that apply

Which of the following is the correct statement?

- **A.** Cookie size can impact the website performance.
- **B.** Cookie size does not affect performance in any practical/meaningful sense.
- **C.** A request to static.example.com will include cookies set for example.com.
- **D.** A request to static.example.com will not include cookies set for example.com.

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	Cookies ride on **every** request to the domain, so their size is multiplied by the number of requests. On a 50-request page a 4 KB cookie becomes 200 KB of upload overhead. The hedged word **"can"** also makes it safe: it does not claim cookies *always* matter, only that they can.
B	FALSE	The direct opposite of A, so exactly one of the pair is right. The phrase "in any practical/meaningful sense" makes the claim **stronger**, not safer – and the cookie-free-domain optimisation exists precisely because the effect *is* practical.
C	TRUE	A cookie "set for example.com" means one whose **domain is** `example.com` . A cookie with that domain applies to `example.com` **and all its subdomains**, so `static.example.com` receives it.
D	FALSE	The opposite of C, and the pair rule applies again.

ANSWER TO QUESTION 17

A and C

AN HONESTY NOTE: THIS OPTION IS WORDED LOOSELY, AND YOU SHOULD KNOW WHY

Strictly, whether a cookie reaches a subdomain depends on the `Domain` attribute:

- `Set-Cookie: sid=abc; domain=example.com` › sent to `example.com` **and** `static.example.com` .
- `Set-Cookie: sid=abc` with **no** Domain attribute › **host-only**, sent to `example.com` alone.

This question does not say which was used, so on the strictest reading it is ambiguous. Two reasons the intended answer is still **C**:

1. The phrase "cookies *set for* example.com" most naturally means "cookies whose domain is example.com", which is the first case.

2. C and D are a direct opposite pair, so exactly one must be true – and the version that makes the question meaningful, and matches the standard cookie-free-domain teaching point, is C.

Exam tip: the same question in other papers includes the clarifier *"(assuming domain attribute is specified with example.com)"*. If you see that bracket, the answer is definitely C. If the bracket is missing, still answer C – but now you know exactly what the bracket was doing.

PART D3 • QUESTION 26

The Git workflow

D3.1 Git has four places your work can live

STORY BOX • FROM DESK TO FILING CABINET TO HEAD OFFICE

Your desk (working directory). Where you actually scribble and edit. Messy, and nothing here is recorded yet.

The out-tray (staging area). You pick up *only the pages you want to file* and place them in the tray. Choosing pages is `git add`. Pages left on the desk are not going anywhere.

Your own filing cabinet (local repository). You staple the tray's contents together, write a note on the front explaining what it is, and file it. That is `git commit -m "..."` – and the note is why `-m` exists.

Head office (remote repository). You post a copy of the folder to head office so colleagues can see it. That is `git push`.

Branches are parallel drafts of the same document, so you can work on a new idea without disturbing the version everyone else relies on.

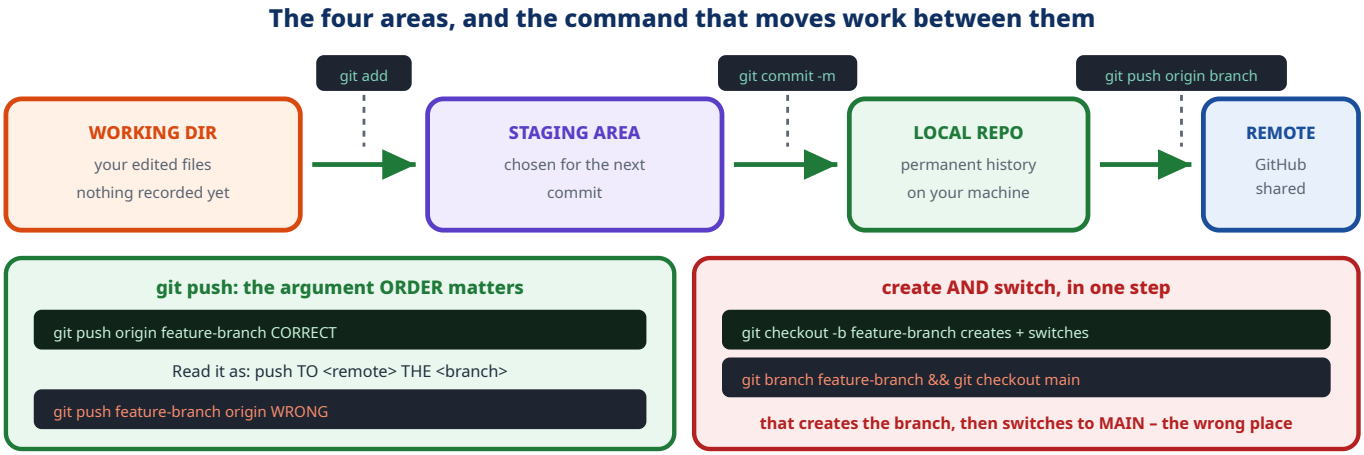

Figure D3.1 – *The `-b` flag in `git checkout -b` is what makes it create the branch as well as switch to it.*

Question 26

select all that apply

Hamza performs these operations: (1) creates a new branch and switches to it, (2) stages changes for commit, (3) commits changes with a message, (4) pushes the branch to a remote repository. Which commands are correctly used for these operations?

- **A.** `git checkout -b feature-branch`
- **B.** `git add .`
- **C.** `git commit -m "Initial commit"`
- **D.** `git push origin feature-branch`
- **E.** `git branch feature-branch && git checkout main`
- **F.** `git push feature-branch origin`

Command-by-command reasoning

Option	Verdict	Why
A	CORRECT	Operation 1. The `-b` flag means **create the branch and switch to it** in one command. Exactly what was asked. (`git switch -c feature-branch` is the modern equivalent.)
B	CORRECT	Operation 2. `git add .` stages everything in the current directory tree. The dot is the path argument, meaning "here and below".
C	CORRECT	Operation 3. `-m` supplies the message inline. Without it, Git opens an editor and waits – so `-m` is what makes this "commit **with a message**".
D	CORRECT	Operation 4. The order is `git push <remote> <branch>`, and `origin` is the conventional name for the remote you cloned from.
E	WRONG	**Half right, and that is the trap.** `git branch feature-branch` does create the branch – but `git checkout main` then switches you to **main**, not to the new branch. So you end up creating a branch and *not* being on it, which is the opposite of "switches to it". (`git branch feature-branch && git checkout feature-branch` *would* work – it is just the long way of writing option A.)
F	WRONG	**Arguments reversed.** Git would read `feature-branch` as the name of a remote, look for a remote called that, and fail with something like `'feature-branch' does not appear to be a git repository` .

ANSWER TO QUESTION 26

A, B, C and D

The four commands that carry out the four operations, in order.

TRAP BOX • BOTH WRONG OPTIONS ARE "NEARLY RIGHT"

Neither E nor F is nonsense – each is a real command with a real effect. They fail on a **single detail**: E switches to the wrong branch, F reverses two arguments. Command-line questions are almost always decided this way, so read arguments **left to right and out loud**: *"push – to origin – the branch feature-branch."* If the sentence does not make sense, the command is wrong.

EXTRA MARKS: THE COMMANDS EXAMINERS PAIR WITH THESE

git status	what is modified, staged, untracked
git add file.py	stage one specific file rather than everything
git commit -am "msg"	stage all **tracked** changes and commit in one step (misses new files)
git checkout main	switch to an **existing** branch (no `-b`)
git branch	list branches; the current one is starred
git push -u origin branch	push and remember the upstream, so later `git push` alone works
git pull	fetch and merge the remote's changes into your branch
git merge feature-branch	bring a branch's work into the one you are on
git log --oneline	compact history

PART E

Revision

The Trap Museum, six cheat sheets, and practice questions built from the same ideas. This is the part to reread the night before.

PART E1

The Trap Museum

Every wrong option in this paper was **designed**. Once you see the small number of patterns behind them, you can spot them in an unseen paper too. Here is every trap this paper set, grouped by type.

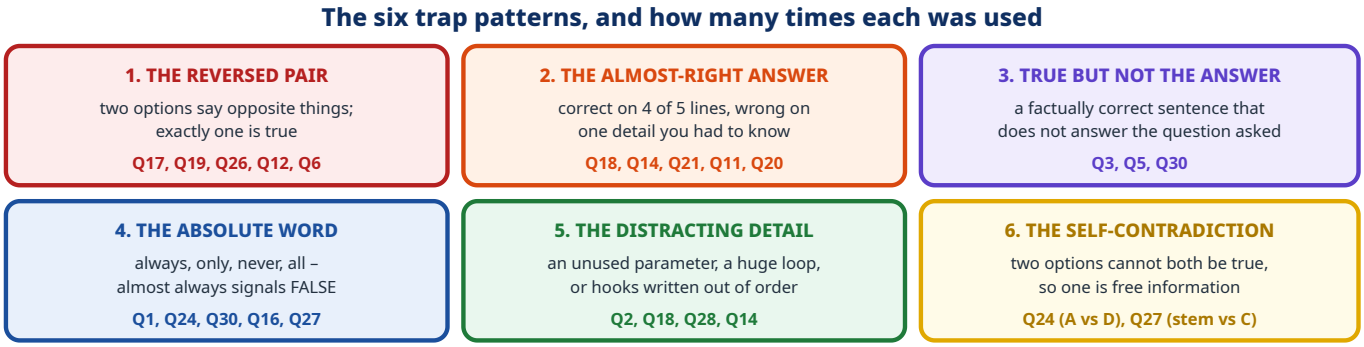

Figure E1.1 – *Six patterns cover every distractor in a 32-question paper.*

E1.1 The complete trap list, question by question

Q	The trap	What it wanted you to believe, and the truth
1	"owns the function"	That a method belongs to its object forever. **Truth:** `this` is decided by the **call site**, not by where the function was written.
2	unused parameters	`idx` and `array` were declared to tempt you into adding `idx`, giving 9. **Truth:** unused parameters change nothing. Answer 6.
3	a true disadvantage	"JWTs are longer" is **true**, but the question asked for an **advantage**.
5	`import * as stats`	Valid code that *loads* the module – but it creates `stats`, so the names `strikeRate` and `playerName` do not exist on their own.
7	both handlers run	That `resolve` then `reject` prints twice. **Truth:** a promise settles **once**; the second call is silently ignored.
8	push vs replace	That the mutation replaces the array, giving `['React']`. **Truth:** `push` appends to the existing array.
9	blank page	That an unmatched route shows nothing. **Truth:** the `'*'` wildcard catches it and redirects.
10	"== is loose, so true"	Option C gets both `typeof` answers right, then guesses `NaN == null` is true. **Truth:** NaN equals nothing; `null` only loosely equals `undefined`.

11	"This is a Dog Labrador"	That an inherited method uses the child's extra data. **Truth:** overriding decides *which* method runs; the method's own **body** decides what it prints, and `getType` only mentions `type`.
12	a phantom `city` key	Option C gets `rest` right, then adds `city` to `newPerson`. **Truth:** destructuring **removed** `city` from `rest`, so nothing can put it back.
13	which requests hit the cache	15 s = ignoring the cache; 5 s = assuming **user 2** also hits it. **Truth:** `memoize` keys on arguments, so user 2 is a genuine miss. 10 s.
14	`reject` hidden after a long loop	The arithmetic is satisfying and correct, so you relax and tick "Passed". **Truth:** the last line is `reject`, so it is **Failed** [75,75,63].
15	bind vs model	Swapping the one-way `v-bind` with the two-way `v-model`.
16	"better SEO by default"	That SPAs help SEO. **Truth:** the opposite – the first response is nearly empty, which is why server-side rendering exists.
17	domain restricts	That naming a domain narrows the cookie. **Truth:** it **widens** it to all subdomains.
18	the expert's trap	Option C rewards you for knowing the hook order and then applying **all four** hooks. **Truth:** `beforeCreate` runs before `data` exists, so its changes are lost. 14 becomes **9**.
19	401 and 403 swapped	**Truth:** 401 = I do not know you (no token). 403 = I know you and you may not (wrong role).
20	"spread makes a full copy"	That `{...item}` protects `item.stats`. **Truth:** spread is **shallow**, so the nested `stats` object stays shared.
21	"100 undefined 100"	That a plain method keeps its object when detached. **Truth:** `func3` was detached on the `const` line and loses `this` too.
22	the threaded default	That Flask is single-threaded by default, giving 20, 50. **Truth:** `app.run()` is **threaded**, so 20, 40.
23	missing `$`, missing `.state`, wrong verb	Each wrong option breaks exactly one rule. **Truth:** `this.$store.state.packed` and `commit` for a mutation.
24	"only a single source"	**Truth:** per-field resolvers mean a query can combine many sources – a headline GraphQL feature.
25	token in the body	**Truth:** it goes in the `Authorization: Bearer` **header**.
26	two nearly-right commands	E creates the branch then switches to **main**; F reverses `push`'s arguments.
27	"Vue blocks all rendering"	Contradicted by the stem itself – the sidebar appeared instantly, so rendering was not blocked.
28	"multiple times due to the loop"	Confusing **work inside one call** with **number of calls**. The `console.log` is outside the loop.
30	"always faster", "runs only once"	Both overstate. **Truth:** cached **per dependency change** – not always faster, not only once.
31	"destroyed and recreated"	**Truth:** same route pattern means the component is **reused**, so `updated` fires and `created` does not.

32	"regardless of reactivity"	**Truth:** `updated` is a **consequence** of reactivity, not independent of it.

PART E2

Six cheat sheets

1. JavaScript values and equality

- **Primitives** (number, string, boolean, undefined, null, symbol, bigint) are **copied**. **Objects** (including arrays and functions) are shared by **reference**.
- `typeof null` is **"object"** – a language bug. `typeof []` is **"object"**. `typeof NaN` is **"number"**.
- `NaN` is equal to **nothing**, including itself. Test with `Number.isNaN(x)`.
- `null` is loosely equal to **undefined only**. `null == 0` is false.
- `{...obj}` is a **shallow** copy: nested objects stay shared. For a deep copy use `structuredClone(obj)`.
- Destructuring with `...rest` **removes** the names you listed from `rest`.
- `location: city` **renames** – the old key is gone.

2. Functions, `this`, classes

- **Regular function:** `this` is decided at the **call site**. Look immediately left of the dot. No dot means `window` (or `undefined` in strict mode / modules).
- **Arrow function: no own `this`**; it inherits **lexically** from where it was written, and freezes it at creation time. `call` and `bind` cannot change it.
- Priority order for `this`: **new** › **explicit** (call/apply/bind) › **method** (the dot) › **default**.
- Detaching a method (`const f = obj.m`) loses `this`.
- **Overriding:** the search stops at the first prototype that has the method. But the method's **body** decides what it outputs.
- `instanceof` searches the **whole** prototype chain, so a Dog is also an Animal.
- `Child.prototype.__proto__ === Parent.prototype` is exactly what `extends` creates.

3. Promises, async and modules

- A promise settles **once**. A later `resolve` or `reject` is **silently ignored**.
- `.catch` runs only if the promise **rejected** or an earlier `.then` **threw**.
- `await` on a rejected promise **throws**, so `try/catch` catches it. The `catch` parameter is **whatever you passed to `reject`** – it need not be an Error.
- The `new Promise` executor runs **immediately and synchronously**.
- `reduce((acc, val) => ..., initial)`: whatever you return becomes the next `acc`. Always pass an initial value.
- Named exports need **braces**: `import { a } from './m.js'`. Default export takes **no** braces. `import * as ns` gives you `ns.a`.
- `require` is CommonJS and does not mix with ES module syntax.

4. Vue

- **Directives:** `v-if` conditional • `v-for` list • `v-bind` attributes (one-way) • `v-model` two-way.
- **Lifecycle order:** beforeCreate › created › beforeMount › mounted. **Not** the order written in the file.
- **In `beforeCreate`, `data` does not exist yet**, so changes are lost. The first usable hook is `created` – hence it is where API calls go.
- **computed** is cached until a **dependency** changes; **methods** re-run on every render. Computed cannot take arguments.
- Changing data that a computed property does **not** read will **not** re-run it.
- **Router:** `path: '*'` is the catch-all in Vue Router 3 and must come **last**.
- **Same route pattern, different parameter › the component is REUSED.** `created` / `mounted` do not re-run; `updated` does. Use a **watcher** to react.
- **Vuex:** `this.$store.state.x` to read • `commit` runs a **mutation** (synchronous) • `dispatch` runs an **action** (may be async). Note the `$`.
- **SPA:** faster navigation and less backend load, but **worse SEO by default**.

5. Flask and backend

- `app.run()` is **threaded by default** (Flask 1.0+). `threaded=False` makes requests queue.
- Timing formula: **(start second + waiting time + own sleep) mod 60**. The clock is read **after** the sleep.
- **`@cache.memoize`** keys on **arguments**. **`@cache.cached`** keys on the **request path** – using it on a parameterised function is a data-leak bug.
- **JWT** = `header.payload.signature`. Payload holds the **claims** (identity, `exp`). It is **signed, not encrypted** – anyone can read it.
- JWT's main advantage is being **stateless**: no server-side session storage.
- Token travels in `Authorization: Bearer <token>`, a **header**.
- **401** = not authenticated ("I do not know you"). **403** = authenticated but not authorised ("I know you, and no").
- **GraphQL:** schema of types and fields, plus a **resolver function per field**. `query` reads, `mutation` writes. One endpoint, normally POST. Can aggregate **many** sources.

6. Platform and tooling

- **WebSocket handshake:** an HTTP **GET** with `Upgrade: websocket`, answered **101 Switching Protocols**. Then it is `ws://` and full duplex.
- **Cookies** are attached to **every** request for the domain, so size matters. Fix: a **cookie-free domain** for static assets.
- `domain=example.com` **widens** the cookie to all subdomains. Omit the attribute to keep it host-only.
- **Git:** working dir › `git add` › staging › `git commit -m` › local repo › `git push origin branch` › remote.
- `git checkout -b name` = create **and** switch. Without `-b` it only switches to an existing branch.
- `git push <remote> <branch>` – remote first. Reversing it fails.

PART E3

Practice questions

These are new questions built from the same ideas, in the same style. Cover the answers. If you can do all twelve, you have genuinely learned this paper rather than memorised it.

P1. What does this print? `console.log(typeof [], typeof null, [] == false, [] === false)`

P2. Predict the output:

```
const a = { x: 1, inner: { y: 2 } };
const b = { ...a };
b.x = 99;
b.inner.y = 99;
console.log(a.x, a.inner.y);
```

P3. What is logged?

```
const o = { v: 7, get: function(){ return () => this.v } };
const g = o.get();
const h = o.get;
console.log(g(), h()());
```

P4. A promise calls `reject('bad')` and then `resolve('good')`. It is used as `p.then(console.log).catch(console.log)`. What prints?

P5. `[5, 10].reduce((acc, val, idx) => acc + val + idx, 100)` – what is the result? Show your working.

P6. A Vue component has `data: { n: 1 }`, `beforeCreate() { this.n = 50 }` and `created() { this.n = this.n + 5 }`. What does `{{ n }}` render?

P7. A computed property `total()` reads only `this.price`. A button sets `this.clickCount++`. After five clicks, how many times has `total()` been evaluated?

P8. A Vuex store has a mutation `SET_USER` and an action `loadUser`. Write the two component lines that (a) read `state.user` in a computed property and (b) trigger the **action**.

P9. A Flask route sleeps 20 s and another sleeps 5 s. Both are fetched at 09:00:50 with `app.run(threaded=False)`, the 20 s route requested first. What two numbers are logged?

P10. With `@cache.memoize(timeout=60)` on `def f(x)` that sleeps 3 s, requests arrive for x = 7, 7, 8, 7 within one minute. Total time?

P11. An endpoint has `@auth_required('token')` and `@roles_required('admin')`. A user with a valid token and role `'editor'` calls it. Status code? What if the same user sends no token at all?

P12. Which Git command creates a branch called `fix`, switches to it, and then pushes it to GitHub? Write both commands.

ANSWERS TO THE PRACTICE QUESTIONS

P1. object object true false. `typeof []` is "object"; `typeof null` is "object". `[] == false` is **true** because loose equality converts both sides to `0`. `[] === false` is **false** because strict equality compares types first, and object is not boolean.

P2. 1 99. `b.x = 99` touched only `b`, because `x` is a top-level primitive that was genuinely copied. But `b.inner` is the **same object** as `a.inner`, since spread is shallow – so `a.inner.y` became 99 too.

P3. 7 undefined. For `g`: `o.get()` was called with the dot, so `this` was `o`, and the arrow froze that – giving 7. For `h()()`: `h` is detached, so the outer call has no owner and the arrow freezes `window`; `window.v` is `undefined`.

P4. bad. The `reject` came **first**, so the promise settled as rejected and the later `resolve` is ignored. `.then` is skipped and `.catch` prints `bad`. (The mirror image of Q7 – same rule, reversed order.)

P5. 116. Start acc = 100.
idx 0: 100 + 5 + 0 = 105.
idx 1: 105 + 10 + 1 = **116**.
This is the trap Q2 set up but did not use – here `idx` really is in the body.

P6. 6. The `beforeCreate` assignment is **lost** because `data` does not exist yet. So `created` starts from the declared `n = 1` and gives 1 + 5 = 6. If `beforeCreate` had worked you would have got 55 – which is exactly the wrong answer Q18 was fishing for.

P7. Once. `clickCount` is not a dependency of `total()`, since the function never reads it. The cache is never invalidated, so after five clicks `total()` has still only been evaluated for the initial render.

P8. (a) `user() { return this.$store.state.user }` inside `computed`. (b) `this.$store.dispatch("loadUser")` – **dispatch**, because `loadUser` is an **action**. Note that `SET_USER` would instead be `commit`ted, and the capitals are the clue.

P9. 10, 15. With `threaded=False` the requests queue. The 20 s route runs first: 09:00:50 + 20 s = 09:01:10, so it logs **10**. The 5 s route can only start at t = 20 and finishes at t = 25 = 09:01:15, so it logs **15**. Note both wrapped past 59 – remember the `mod 60`.

P10. 6 seconds. x = 7 is a miss (3 s). The second x = 7 is a **hit** (0 s). x = 8 is a different key, so a miss (3 s). The final x = 7 is a hit (0 s). 3 + 0 + 3 + 0 = 6.

P11. 403 with the valid token, because identity was established but the role `'editor'` does not match `'admin'`. With **no token** it is **401**, because the first gate fails and the role is never even considered.

P12. `git checkout -b fix` then `git push origin fix` (or `git push -u origin fix` to set the upstream so later pushes need no arguments).

PART E4

Final word

THE TWELVE SENTENCES THAT COVER MOST OF THIS PAPER

1. `typeof null` is **"object"**, `typeof NaN` is **"number"**, and **NaN equals nothing**.
2. **Spread is shallow.** Nested objects stay shared.
3. Destructuring with `...rest` **removes** the keys you named.
4. For a regular function, `this` is decided by the **call site**. An **arrow freezes** `this` **at creation**.
5. **Overriding** picks the method; the method's **body** picks the output.
6. A promise **settles once**; `await` turns a rejection into a **throw**, and you can reject with any value.
7. **Named exports need braces.**
8. Vue hooks run **beforeCreate, created, beforeMount, mounted** – and `beforeCreate` **cannot touch** `data` .
9. **computed is cached per dependency**; a change it does not read will not re-run it.
10. Same route pattern means the component is **reused**, so `updated` fires and `created` does not.
11. `app.run()` is **threaded**; `memoize` keys on **arguments**; **401** is "who are you", **403** is "not you".
12. WebSocket handshakes with **GET**; cookies with a `domain` reach **subdomains**; `git checkout -b` creates **and** switches.

HOW TO USE A PREVIOUS-YEAR PAPER PROPERLY

1. **Attempt it closed-book first**, timed. A paper you read the answers to teaches you very little.
2. **For every question you got wrong, write down the single fact you were missing** – not the answer. "beforeCreate cannot touch data" is a fact you can reuse; "the answer was D" is not.
3. **Then read the Trap Museum** in Part E1 and find your mistake's pattern. Most students make the same *type* of mistake repeatedly.
4. **Re-derive the code questions on paper**, drawing the boxes and tables from this book. Speed comes from having a method, not from recognition.
5. **Do the practice questions in Part E3 a week later.** If you can still do them, the knowledge is yours.

Prepared as a concept-first solution guide to a previous year question paper.
Every JavaScript answer here was verified by executing the code.

NOTES ON SOURCES AND VERIFICATION

Verified by execution: the eight JavaScript output questions (Q2, Q7, Q10, Q11, Q12, Q14, Q20, Q21) were run in Node.js and the console output reproduced in this book verbatim.

Checked against documentation: Vue 2 lifecycle order and the fact that reactive `data` is installed after `beforeCreate`, computed-property caching, and Vue Router component reuse (v2.vuejs.org); Vuex `commit` versus `dispatch` (vuex.vuejs.org); Flask's `threaded=True` default for `app.run()` and Flask-Caching's `cached` versus `memoize` key rules (flask.palletsprojects.com, flask-caching.readthedocs.io); JWT structure (RFC 7519) and Flask-JWT-Extended token location; HTTP status semantics for 401 and 403 and the WebSocket opening handshake (MDN, RFC 6455); cookie `Domain` attribute behaviour (RFC 6265); GraphQL types, fields and resolvers (graphql.org/learn); and Git command semantics (git-scm.com/docs).

All explanations, analogies, diagrams, worked traces and practice questions here were written and drawn for this document. Content was rephrased for compliance with licensing restrictions.